Piranha: An Autonomous Water Surface Robot

A thesis submitted to the
Graduate School
of the University of Cincinnati
in partial fulfillment of the
requirements for the degree of

Master of Science

in the Department of Aerospace
Engineering & Engineering Mechanics
by

Xiahua Liu

B.S. Beijing Institute of Technology
June, 2018

Committee Chair: Ou Ma, Ph.D.

Abstract

This thesis describes the design and development work of a marine debris cleanup system - Piranha. It is an autonomous water surface vehicle that can collect different types of garbage such as plastic bottles and bags. To begin with, Piranha is simple, with minimum moving parts including only two thrusters and a lever system. Necessary sensors like IMUs and GPS on Piranha enable positioning and navigation functions. Besides sensors, a LoRa wireless module allows the human worker to control Piranha from a distance. The simulation software for Piranha is discussed in chapter four with the theoretical equations and numerical methods. In the end, some control algorithms, including a 13 state EKF, a heading controller, and a trajectory tracking controller, are brought up to achieve autonomy at a certain level. This report reveals the development cycle of Piranha, from the very simple low-level mechanical parts to the high-level robotics system design.

Acknowledgements

First of all, I want to thank the project founder, Michael Arens. He provided the idea and named this project. Without him, the project would not exist.

During the control system design, Jonathan and Krishna provided their opinions and suggestions to me. However, it almost ended up in a fight because we could not agree with each other. As a result, I combined their ideas, meanwhile absorbed the advantages of different existing techniques into the final control system design.

Byron Castillo, currently a Mechanical Engineering student at the University of Cincinnati, finishes the mechanical design part. Benjamin Russ is the industrial designer for the product, and he also took part in the mechanical design of Piranha.

In the end, I also want to thank the University of Cincinnati 1819 Innovation Hub and Ben Jones, the Makerspace Manager. They provided the workspace for us and also funded this project.

During my two years of study at the University of Cincinnati, Prof. Ou Ma, Prof. Donghoon Kim, Prof. Janet Dong, and Prof. Shaaban Abdallah in the College of Applied Engineering and Science had generously shared the professional knowledge with me. This invaluable knowledge and experience helped all the work done in this project.

Contents

List of Figures **vi**

List of Tables **ix**

1 Introduction **1**
1.1 Background . . . 1
1.2 Plastic harms . . . 2
1.3 Plastics on the shorelines . . . 3
1.4 Trash collection systems . . . 5
1.4.1 Venturi Pump . . . 5
1.4.2 Conveyor . . . 6
1.4.3 Mesh bucket . . . 8
1.5 Similar Robots . . . 9
1.5.1 WasteShark . . . 9
1.5.2 Clearbot . . . 9

2 Mechanical Design **11**
2.1 Floats . . . 12
2.1.1 Float Assembly . . . 14
2.2 Thrusters . . . 15
2.2.1 Thruster Locations . . . 16

3 Electrical System **18**
3.1 Power Module . . . 20
3.1.1 MOSFET . . . 22
3.1.2 Thermal Estimation . . . 23
3.1.3 Switch system . . . 24
3.1.4 Voltage Measuring Point . . . 24
3.2 Battery System . . . 24
3.3 Motors (BLDC) . . . 25
3.4 Electric Speed Controller (ESC) . . . 26
3.5 MCU . . . 26
3.6 IMU . . . 28
3.7 Magnetometer . . . 31
3.8 GPS . . . 32
3.9 Wireless Module . . . 33

4 Simulation **35**
4.1 Assumptions . . . 36
4.2 6 DoF State-Space Equations of Motion . . . 36
4.3 Reference Frames . . . 37
4.3.1 Transform Vectors from Local NED Frame to Body Frame . . . 38
4.4 Mass Matrix . . . 39
4.5 Fluid Dynamical Model . . . 40
4.5.1 Hydrostatic analysis . . . 41
4.5.2 Hydrodynamic analysis . . . 43
4.6 Numerical Calculation . . . 43
4.6.1 Mesh Simplification . . . 43
Buoyancy Estimate . . . 44
Drag & Skin Friction Estimate . . . 49
Wind and Current Vectors . . . 51
4.7 Input Mapping . . . 51
4.8 Simulation Results . . . 53
4.8.1 Drop Test . . . 53
4.8.2 Linear Driving Test . . . 55
4.8.3 Steering Test . . . 57

5 Control System **60**
5.1 Navigation Model . . . 60
5.1.1 Equation of Motion . . . 61
5.1.2 Control Input . . . 61
5.1.3 Mass matrix . . . 63
5.1.4 Damping Matrix . . . 63
5.2 Extended Kalman Filter . . . 64
5.2.1 States and Observations . . . 64
5.2.2 State Predict . . . 65
IMU Prediction Equation . . . 65
5.2.3 Attitude Update . . . 67
Body Magnetic Field . . . 68
5.2.4 Position Update . . . 68
5.2.5 EKF Results . . . 70
5.3 PD Controller for Yaw Control . . . 71
5.3.1 Test Results . . . 72
5.4 Trajectory Tracking . . . 77
5.4.1 Feedback Linearization Controller . . . 78
Circle Trajectory Result . . . 80
Square Trajectory Result . . . 83

6 Conclusion **87**
6.1 Future work . . . 88

A BlueRobotics Thruster T200 Data **89**

B Piranha EKF SymPy Script **92**

Bibliography **96**

List of Figures

1.1 Some fishes lives with plastic wastes in water. . . . 2
1.2 Piranha rendered model, only the floats and frames are shown. . . . 4
1.3 Piranha during the forth pond test in July, 2021. . . . 5
1.4 A simplified model of Venturi pump collection system. We fixed the capture net near the dotted lines. Pink stars represent the floating trashes in water. The pressure drop along the channel forces water to flow because of the Venturi effect. . . . 6
1.5 A Conveyor collection system illustration. The upper roller is motorized; meanwhile, the lower roller is passive. Pink stars and the green bucket represent the trashes to be collected and the trash container, respectively. . . . 7
1.6 A concept model of Piranha using the conveyor collection system. . . . 7
1.7 On the left part, the Piranha is driving forward. On the right part, Piranha is moving backward. . . . 8
1.8 Clearbot. . . . 10

2.1 A rendered model of Piranha. The white blocks represent the electrical system box and the battery pack. . . . 12
2.2 Pontoon nose cone. . . . 13
2.3 Pontoon body. . . . 14
2.4 The overall float assembly. . . . 15
2.5 BlueRobotics T200 Thruster (Isometric View). . . . 16
2.6 BlueRobotics T200 Thruster (Back View). . . . 16
2.7 Two thrusters are installed at the rear of Piranha. . . . 17

3.1 The electrical system overview. . . . 18
3.2 The figure shows Piranha's test module we built in 1819 Innovation Hub for testing the software functions and electrical design. . . . 19
3.3 The overall schematic of the power module. . . . 20
3.4 Power module rendered 3D view in KiCad. . . . 21
3.5 The PCB prototype for the power board manufactured by JLCPCB. . . . 21
3.6 The voltage divider circuit. J5 is supposed to connect to the analog input pin of the MCU. . . . 24
3.7 I took the picture at an early stage test. The battery packs on the left are glued to the box to prevent collisions. . . . 25
3.8 I calibrated two thrusters with a game controller. The right thruster is spinning. Meanwhile, the left one is not moving. . . . 26
3.9 BlueRobotics Basic ESC. . . . 27
3.10 Pi Pico installed on the breakout board. . . . 27

3.11 Adafruit TDK InvenSense ICM-20948 9-DoF IMU. 29
3.12 Adafruit 9-DOF Orientation IMU Fusion Breakout - BNO085. 30
3.13 WitMotion High-Precision RM3100 Magnetometer Sensor 31
3.14 E32-915T30D 34

4.1 The mass matrix values of Piranha generated in the SOLIDWORKS Mass Properties menu. 40
4.2 Hydrostatic model of Piranha. 41
4.3 On the left is the original mesh generated by SOLIDWORKS, on the right is the simplified mesh model. 44
4.4 A_1 and A_2 are the projected area of the upper part and the lower part on the plane P. 45
4.5 Two partially submerged cases. 47
4.6 The offset water volume is between the two triangles. The face directions for the two triangles are different. 48
4.7 In the mesh model, the area of a triangular face equals the length of its normal vector. 48
4.8 Piranha's hydrostatic estimator flowchart. 49
4.9 Cell level dynamics, drag and skin friction. 50
4.10 Piranha's hydrodynamic estimator flowchart. 50
4.11 BlueRobotics T200 Thruster input-output curve according to the lab test. 52
4.12 Simulation: Drop test result - Positions x, y, z 54
4.13 Simulation: Drop test result - Attitude ϕ, θ, ψ 54
4.14 Simulation: Drop test result - Linear Speed u, v, w 55
4.15 Simulation: Drop test result - Angular Speed p, q, r 55
4.16 Simulation: Linear driving test result - Positions x, y, z 56
4.17 Simulation: Linear driving test result - Attitude ϕ, θ, ψ 56
4.18 Simulation: Linear driving test result - Linear Speed u, v, w 57
4.19 Simulation: Linear driving test result - Angular Speed p, q, r 57
4.20 Simulation: Steering test result - Positions x, y, z 58
4.21 Simulation: Steering test result - Attitude ϕ, θ, ψ 58
4.22 Simulation: Steering test result - Linear Speed u, v, w 59
4.23 Simulation: Steering test result - Angular Speed p, q, r 59

5.1 Piranha navigation model and frame. 61
5.2 Thrusters' location with respect to the mass center. 62
5.3 The overall EKF flowchart on Piranha. 65
5.4 EKF estimated trajectory versus raw GPS data in the linear driving simulation. . . . 70
5.5 EKF estimated trajectory versus raw GPS data in the steering simulation. 70
5.6 Piranha PD heading controller flowchart. 71
5.7 Simulation: Without the PD controller - Positions x, y, z. 72
5.8 Simulation: Without the PD controller - Positions ϕ, θ, ψ. 73
5.12 Simulation: With the PD controller - Positions x, y, z. 73
5.9 Simulation: Without the PD controller - Linear Speed u, v, w. 74
5.13 Simulation: With the PD controller - Positions ϕ, θ, ψ. 74
5.10 Simulation: Without the PD controller - Angular Speed p, q, r. 75
5.14 Simulation: With the PD controller - Linear Speed u, v, w. 75
5.11 Simulation: Without the PD controller - Inputs u_l, u_r. 76
5.15 Simulation: With the PD controller - Angular Speed p, q, r. 76
5.16 Simulation: With the PD controller - Inputs u_l, u_r. 77

5.17 Piranha's trajectory tracking feedback linearization controller flowchart. 78
5.18 Feedback linearization trajectory tracking controller test result, the reference is a circle trajectory. 81
5.19 Feedback linearization trajectory tracking controller test result - Position x, y, z. . . 81
5.20 Feedback linearization trajectory tracking controller test result - Attitude ϕ, θ, ψ. . 82
5.21 Feedback linearization trajectory tracking controller test result - Linear Speed u, v, w. 82
5.22 Feedback linearization trajectory tracking controller test result - Angular Speed p, q, r. 83
5.23 Feedback linearization trajectory tracking controller test result - Input u_l, u_r. 83
5.24 Feedback linearization trajectory tracking controller test result, the reference is a square trajectory. 84
5.25 Feedback linearization trajectory tracking controller test result - Position x, y, z. . . 84
5.26 Feedback linearization trajectory tracking controller test result - Attitude ϕ, θ, ψ. . 85
5.27 Feedback linearization trajectory tracking controller test result - Linear Speed u, v, w. 85
5.28 Feedback linearization trajectory tracking controller test result - Angular Speed p, q, r. 86
5.29 Feedback linearization trajectory tracking controller test result - Input u_l, u_r. 86

A.1 T200 Public Performance Data 10-20V, 12 V 89
A.2 T200 Public Performance Data 10-20V, 14 V 90
A.3 T200 Public Performance Data 10-20V, 16 V 90
A.4 T200 Public Performance Data 10-20V, 18 V 91

List of Tables

2.1 Specifications of 17" Diameter Pontoon Nose Cones 13
2.2 Specifications of 17" Diameter Pontoon Nose Cones 14
2.3 Physical Specifications of the BlueRobotics T200 Thruster 15

3.1 Technical Specifications of the Power Board 22
3.2 Technical Specifications of TOSHIBA TK4R3E06PL (part) 23
3.3 Technical Specifications of BlueRobotics Lithium-ion Battery Module (14.8 V, 18 Ah) 25
3.4 Technical Specifications of BlueRobotics T200 Thruster (part) 26
3.5 Technical Specifications of BlueRobotics Basic ESC (part) 28
3.6 Electrical Specifications of Pi Pico (part) 28
3.7 Specification of ICM-20948 (part) 30
3.8 Specification of BNO-085 (part) 31
3.9 Specifications of RM3100 (part) 32
3.10 Specification of Adafruit Ultimate GPS (part) 33
3.11 Wireless Communication Method Comparison 33
3.12 Specifications of E32-915T30D 34

Chapter 1

Introduction

The aggressive development in the industry in the past few centuries brought humankind to an unprecedented advanced stage. We make gas-powered cars allowing us to run faster than any animals on the earth; ships carrying people cruise across the oceans every day, and even spacecraft to escape from Earth and Sun to deep space. But those brilliant achievements came with prices. One of them is the increasing number of wastes in the marine area.

1.1 Background

There is a giant plastic waste island called the Great Pacific Garbage Patch on the earth. Its size is 1.6 million km^2, more than twice the size of Texas (approximate 0.7 million km^2) [8]. This floating waste "continent" is the outcome of the increasing amount of trash produced by industries and individuals in the past centuries and the weak effort of collecting trash in the water.

Compared with the ocean trash problem, coastal pollution is never more severe in history. According to a 2014 research, even in the most remote area on the earth, traces of plastic wastes were found [4]. It means while we enjoy using plastic forks and bottles, they have surrounded our planet silently. As factories and manufacturers increase their production, more and more plastic

wastes will enter the water in the next few decades. Research shows there will be more plastic than fish in 2050 [1]. Figure 1.1 shows the exact situation that plastic wastes are changing the living environment of sea creatures.

Compared with metal materials, plastic materials are superior in weight and durability and are pretty easy to form in any shape. However, this also makes plastic the most stubborn trash on the earth. Some of them cannot be decomposed in hundreds of years [3]. So we must take action on the marine trash collection instead of hoping them go away one day.

Figure 1.1: Some fishes lives with plastic wastes in water.

1.2 Plastic harms

Most of the marine wastes are made of plastics [8], as they are resistant to corrosion, oxidation, and corruption. Plastic wastes can kill sea creatures in three main ways:

- **Plastic ingestion**. According to the research, fishes in the North Atlantic area ingest thousands of tons of plastic wastes every year, most of which are microplastics. These plastic grains cause intestine bleeding, which can lead sea animals to death. [6]

- **Physical harm**. Some sea creatures like turtles and birds do not own the ability to remove the plastic bags wrapped around their heads, and they may die of suffocation. Another example of the physical damage of plastics is the derelict nylon fishnet lines. It can entangle even giant animals like seals, whales and leave them stranded.

- **Block the sun light**. Tiny algae in the shallow water layer rely on the sunlight to grow and reproduce. Floating wastes block the sunlight, in turn reducing the number of algae. As a result, there is not enough food for small sea animals, the bottom residents of the food chain. The reduction in food limits the overall animal population. On the other hand, fewer plants mean less oxygen. The dissolved oxygen in water also drops, making fishes harder to breathe.

1.3 Plastics on the shorelines

There is research pointing out the vast majority of the plastics in the ocean end up washed, or buried under the shorelines, whether dry shorelines, coastal areas, or offshore areas [12]. However, because shorelines are long and most parts are far from human habitats, the trash collection force on shorelines is significantly less than the land. The main reason is the lack of hands and the high price of human labor.

That is why we designed Piranha. Piranha is a water surface robot that can be operated remotely by a human worker or cruise on its own to collect floating trash near the coast. It has a highly advanced control system built similarly to modern drone controllers, and we equipped it with customized control algorithms. On the mechanical side, it has a straightforward design to

achieve lower building costs and higher reliability.

Figure 1.2 shows a rendered model of Piranha made in SOLIDWORKS 2017. Figure 1.3 shows the overall structure of Piranha during the fourth field test. In that test, we tried using a pulley system to lift the collection bin so that human workers could easily dump the trash when the trash bin was full.

Figure 1.2: Piranha rendered model, only the floats and frames are shown.

Figure 1.3: Piranha during the forth pond test in July, 2021.

1.4 Trash collection systems

Trash collection is the core function of Piranha. While Piranha used the simplest way to collect trash, we had several other options in the early development stage. In this section, I will introduce these different kinds of trash collection systems in the hope of inspiring more ideas from the readers.

1.4.1 Venturi Pump

The device shown in Figure 1.4 is a simplified model of a Venturi pump trash collection system. The compressed air has a leftward momentum, pushing the water to flow. As a result, the trash is captured by the water current and trapped by the net near the tube.

The advantage of this design is that there is no physical contact between the moving parts and the trash, so the chance of being stuck by the debris is relatively low. And compressed air, on the other hand, gives the boat driving power, so the thrusters are not needed anymore. These advantages are essential because driving among floating trashes with thrusters is dangerous. Ropes,

plastic bags can easily get caught by the thrusters and stop the motor, which in the end leads to a complete power system failure.

However, the disadvantages of this system are that the complexity of the system and the low efficiency. The air pump needs too much power compared with the underwater propellers. A common airboat has a 3-5 MPG fuel efficiency rating. Given by today's battery technology, it is unrealistic to drive the airboat with electricity instead of gasoline.

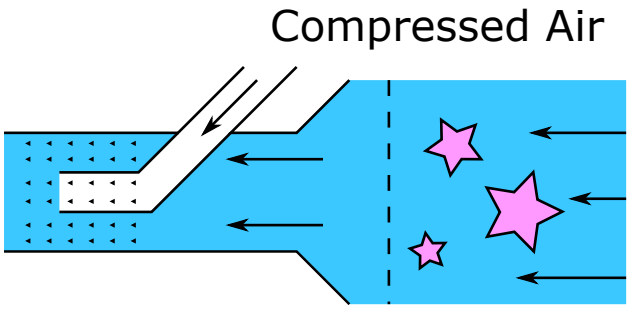

Figure 1.4: A simplified model of Venturi pump collection system. We fixed the capture net near the dotted lines. Pink stars represent the floating trashes in water. The pressure drop along the channel forces water to flow because of the Venturi effect.

1.4.2 Conveyor

The conveyor is the most common solution in collecting floating trashes and algae. Because if we separate the collection part from the trash container, then the trash volume depends on the container only. And the mechanical design is more straightforward compared with the Venturi pump solution. A simple combination of a passive roller and a motorized roller is enough. The conveyor-based trash collection system is shown in Figure 1.5.

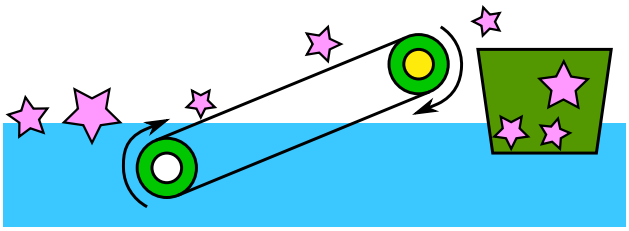

Figure 1.5: A Conveyor collection system illustration. The upper roller is motorized; meanwhile, the lower roller is passive. Pink stars and the green bucket represent the trashes to be collected and the trash container, respectively.

Figure 1.6: A concept model of Piranha using the conveyor collection system.

However, this configuration is usually seen on large-sized boats because there are too many moving parts, and the cost is considerably higher than other options. On Piranha, we did try the conveyor configuration as the model shown in the Figure 1.6. However, due to the high cost and the difficulty in manufacturing, we decided not to adopt this design at the early development stage.

1.4.3 Mesh bucket

The collection bucket is made of meshes and allows water to go through but not trashes. This collection system is passive, meaning the boat needs to carry the bucket and drive around to collect trashes.

This solution is the simplest of all choices, and the cost is low. However, it does have several drawbacks. One of them is the capacity of the mesh bucket limits the total trash carried by Piranha in one run. Second, the bucket is submerged in water, making it very hard for workers to empty the bucket at the dumpsite. Last by most important, because the bucket does not provide any active forces on the already captured trash, there is no guarantee that the garbage will not come out of the bucket if the boat drives backward.

Piranha has a simple lever system to avoid the last two issues. The lever system illustration is shown in Figure 1.7.

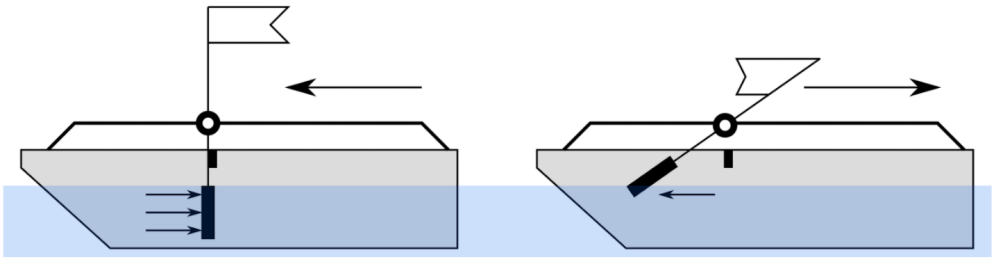

Figure 1.7: On the left part, the Piranha is driving forward. On the right part, Piranha is moving backward.

We carefully designed the lever so that it is nearly perfectly balanced in the vertical position. A minor disturbance will change the angle of the pole. As shown in the left part of the figure, if Piranha drives forward, the rod is vertical because of the water pressure on the lower part. The net's mouth opens in the vertical direction, and trash starts to flow inside the bag.

The right part shows the situation when Piranha drives backward. The lower part starts moving

up due to the water pressure. The mouth of the net leaves the water so that the trash cannot escape. In such a way, the lever automatically "seals" the net when the Piranha starts driving backward or the water current is faster than the boat and is about to bring out the trash.

1.5 Similar Robots

Some other robots can do a similar job as Piranha at the time when we designed Piranha. Almost all of them have one of the configurations mentioned in the previous section. However, they vary in some subtle details when it comes to the goals and technologies behind them.

1.5.1 WasteShark

Sponsored by the Robotics Innovation Center in Germany and the European Union, WasteShark has a very similar design to Piranha. They claimed WasteShark could carry 350-kilogram trash in one run. It also has a special navigation algorithm to allow the robot to return to the dock once the bucket is full. Several water quality sensors on the boat can detect the water quality data, including pH, ORP, conductivity, dissolved oxygen, turbidity, ammonium, chloride, nitrate, salinity, mV, ORP, TDS, resistivity level, and send them back to the data center[15].

1.5.2 Clearbot

Clearbot shown in Figure 1.8 is an intelligent robot designed by a company in Hong Kong. It is very similar to Piranha, except it used a conveyor instead of a mesh bucket. According to the description provided on their website, Clearbot has a self-driving function and a computer and camera to identify the type of garbage and upload the data to a cloud-based platform.

Clearbot's primary function is collecting trash from the water, cleaning the rivers and oceans globally. Besides that, it can also identify a wide range of waste and material types to help the

recycling companies to classify the trashes. According to the recent updates, the developers of Clearbot are working on the swarm algorithm for the robot.

Figure 1.8: Clearbot.

Chapter 2

Mechanical Design

This chapter includes the mechanical details of Piranha. To begin with, Piranha is a twin-body pontoon boat. It has two floats on the left and right sides to support its weight. The only motorized parts on Piranha are the left and right thrusters installed at the rear. In other words, Piranha is a skid steering boat. Both thrusters provide the driving forces forward or backward, allowing Piranha to move forward and back. The difference between the thruster outputs generates the torque required for steering.

Readers can find The overall design of Piranha in Figure 2.1.

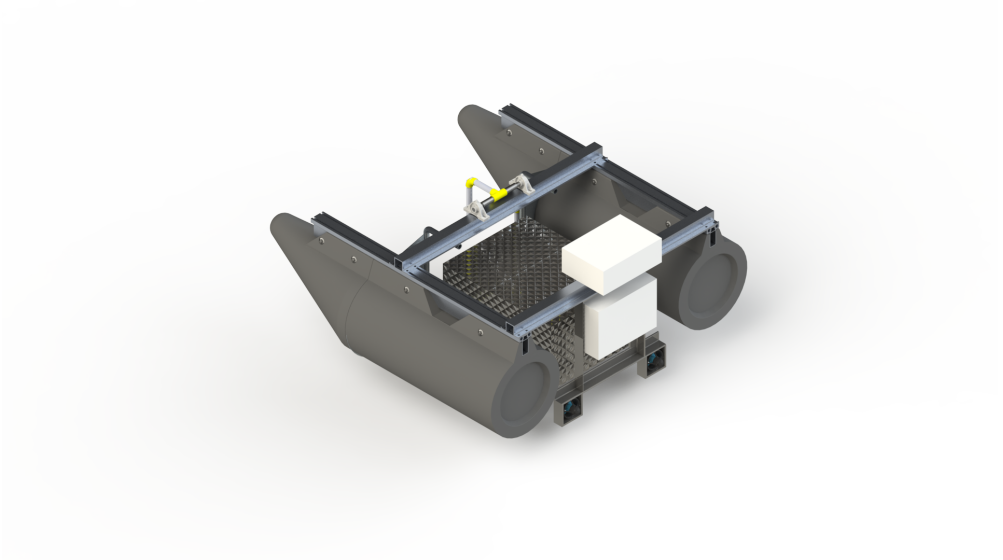

Figure 2.1: A rendered model of Piranha. The white blocks represent the electrical system box and the battery pack.

2.1 Floats

Piranha's floating parts, including the connection frame, are sold by *Silver Lake Fabrication LLC D.B.A Tiny Pontoon Boats*. The floats are made of High-Density Poly Ethylene (HDPE) with closed-cell urethane foam filling. The connection frame is made of aluminum.

The specifications of the pontoon nose part in Figure 2.2, can be found in Table 2.1.

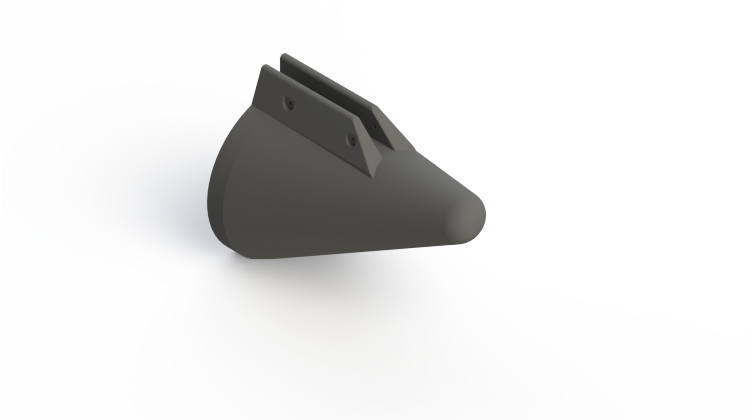

Figure 2.2: Pontoon nose cone.

Table 2.1: Specifications of 17" Diameter Pontoon Nose Cones

Characteristics	**Rating**
Float width	16.5 inches
Float length	25 inches
Weight	14 lbs
Rated capacity	45 lbs

The specifications of the pontoon straight part in Figure 2.3, can be found in Table 2.2.

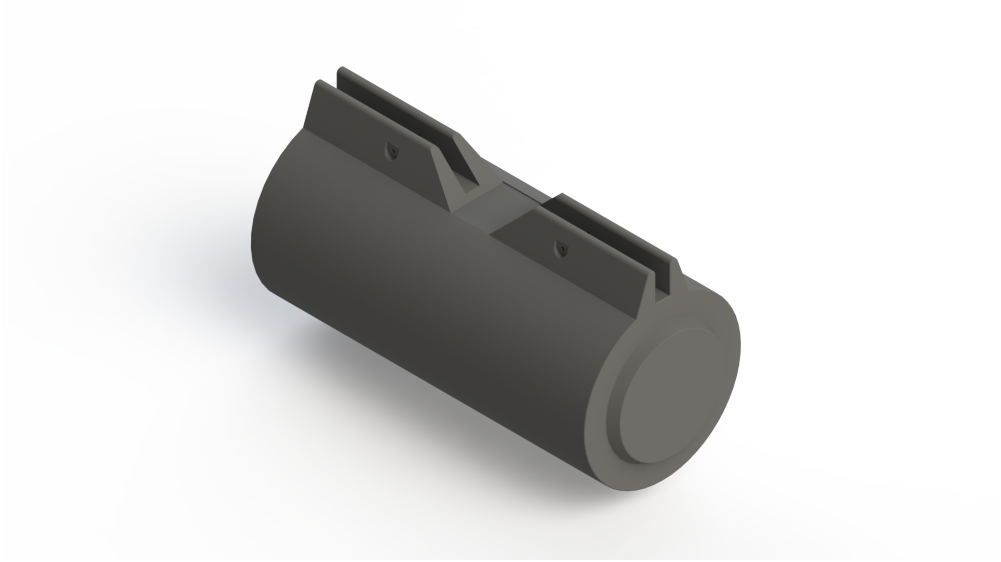

Figure 2.3: Pontoon body.

Table 2.2: Specifications of 17" Diameter Pontoon Nose Cones

Characteristics	**Rating**
Float width	16.5 inches
Float length	36 inches
Weight	23 lbs
Rated capacity	130 lbs

2.1.1 Float Assembly

Piranha connects the float parts by the aluminum frame, shown in Figure 2.4. The screws and nuts are made of stainless steel to prevent corrosion.

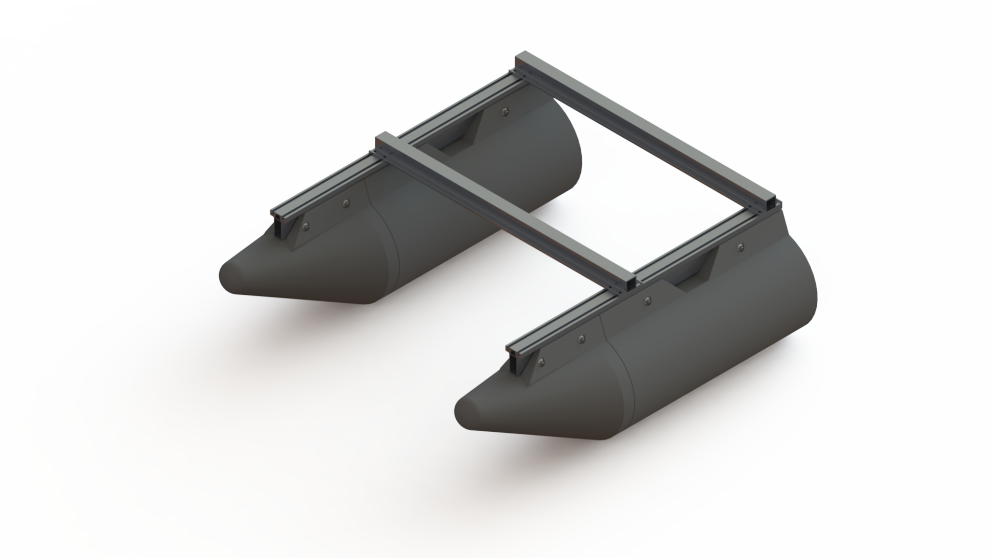

Figure 2.4: The overall float assembly.

2.2 Thrusters

Piranha has two thrusters installed at the rear. The thruster model is BlueRobotics T200 Thruster. Its mechanical drawing can be found in Figure 2.5, 2.6. Table 2.3 lists the specifications for them.

Table 2.3: Physical Specifications of the BlueRobotics T200 Thruster

Characteristics	**Rating**
Width	113 mm
Length	100 mm
Weight	344 g
Propeller Diameter	76 mm

Figure 2.5: BlueRobotics T200 Thruster (Isometric View).

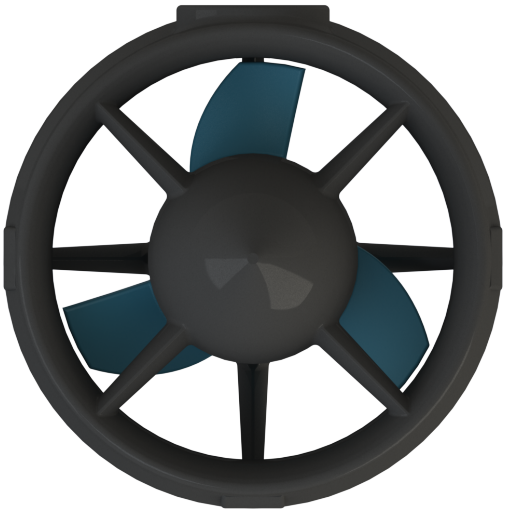

Figure 2.6: BlueRobotics T200 Thruster (Back View).

2.2.1 Thruster Locations

Figure 2.7 shows the location of the two thrusters.

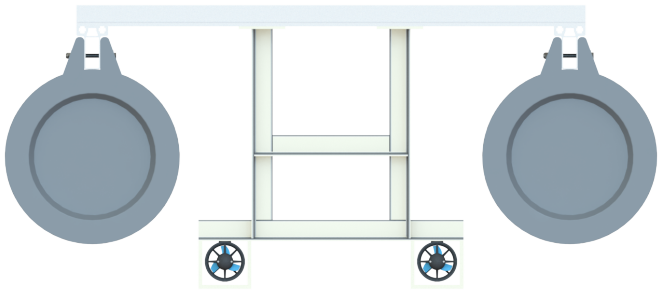

Figure 2.7: Two thrusters are installed at the rear of Piranha.

Chapter 3

Electrical System

Readers can find a simplified schematic of Piranha's electrical system in Figure 3.1. Figure 3.2 shows the real-world test bench.

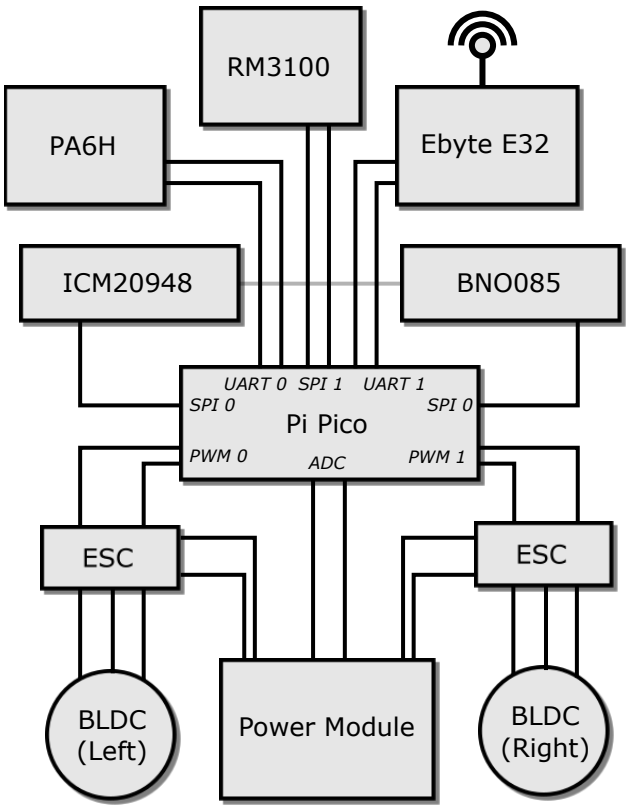

Figure 3.1: The electrical system overview.

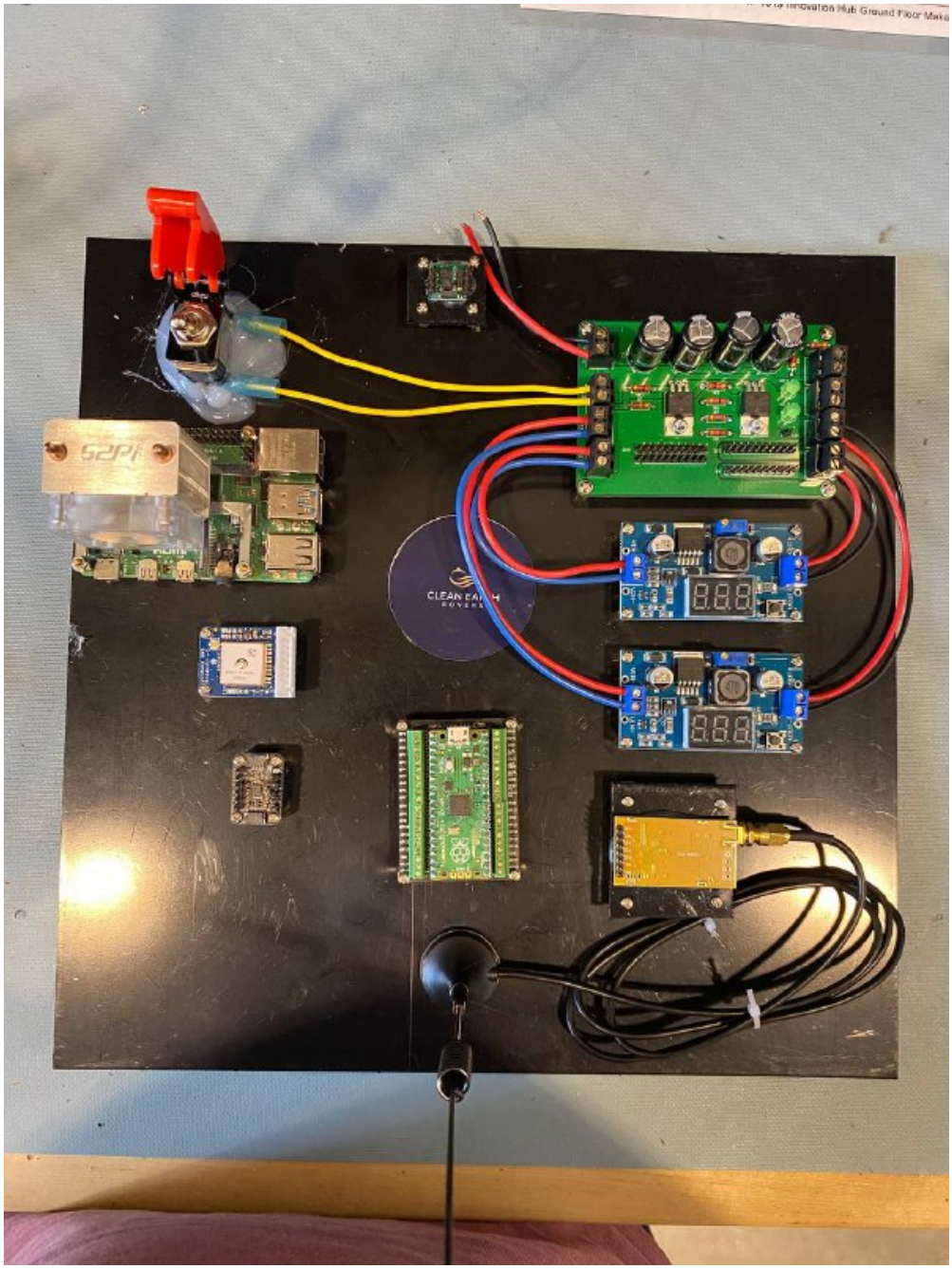

Figure 3.2: The figure shows Piranha's test module we built in 1819 Innovation Hub for testing the software functions and electrical design.

3.1 Power Module

We customized the power module on Piranha to distribute the necessary electrical power to every modules module, including the electrical speed controllers, the Pi Pico, sensors, etc. The board is designed by KiCad and manufactured by JLCPCB. Since the power module carries more than 40-ampere current in the working condition, we covered the PCB with an extra copper layer made by the laser cutter.

The power module also contains several physical switches, including a master switch, two emergency stop switches, and a voltage measurement point for maintenance usage. These switches control the power output to the two thrusters by controlling the gate voltages of two TK4R3E06PL MOSFETs. Table 3.2 lists its specifications.

The schematic and the rendered 3D model of the Piranha's power module are provided in Figure 3.3 and Figure 3.4. A PCB prototype board is shown in Figure 3.5, the electrical components are not soldered yet.

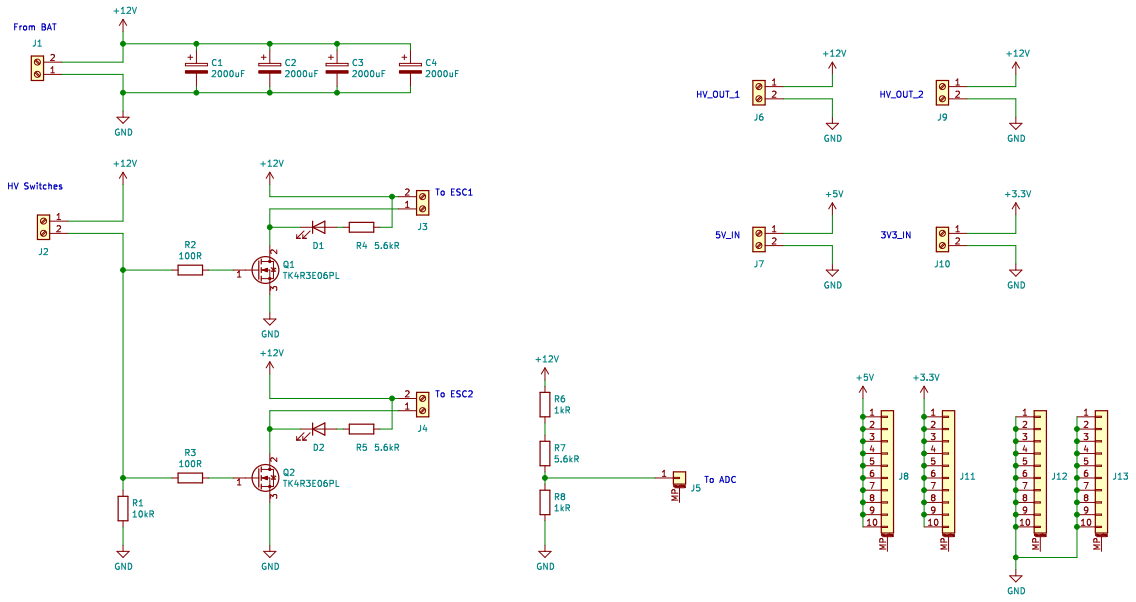

Figure 3.3: The overall schematic of the power module.

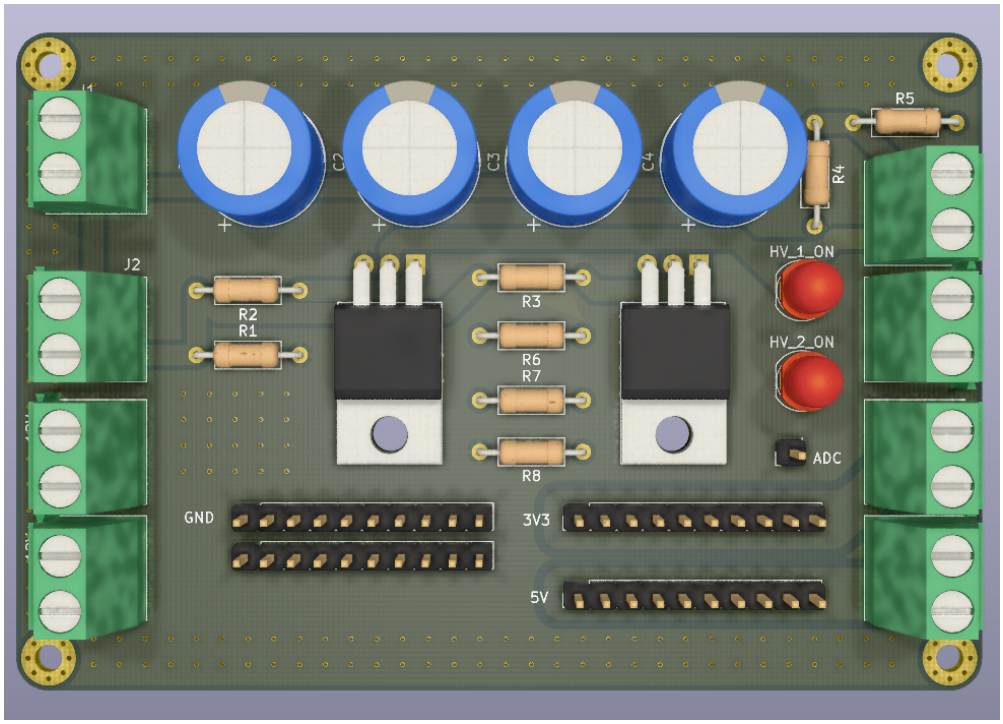

Figure 3.4: Power module rendered 3D view in KiCad.

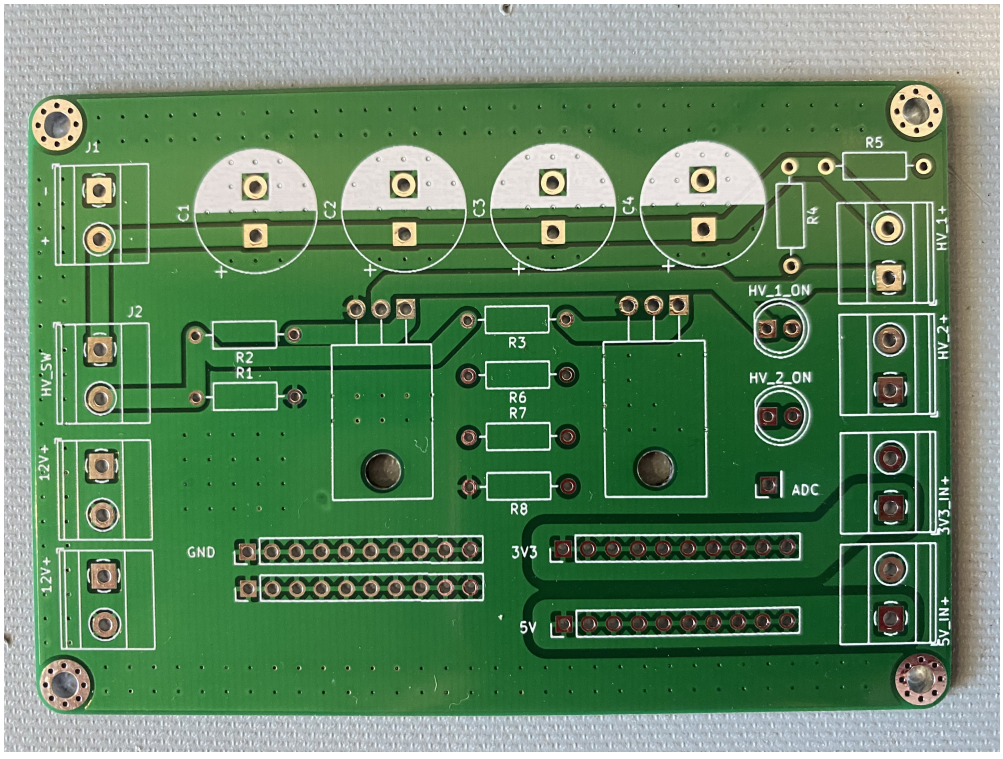

Figure 3.5: The PCB prototype for the power board manufactured by JLCPCB.

Table 3.1 lists the Piranha's power board specification.

Table 3.1: Technical Specifications of the Power Board

Characteristics	**Rating**
Maximum input voltage	25 V
Maximum main output current	2×80 A
Maximum working temperature	125° C
Maximum $3.3V$ output current	3 A
Maximum $5V$ output current	3 A

Please know that this table only describes the capability of the power board itself. It does not reflect the whole system limitation. For example, the maximum current consumed by a thruster is only 24 A.

3.1.1 MOSFET

The power module has two Toshiba TK4R3E06PL MOSFETs as the primary relays. You can find part of its technical specifications in Table 3.2. These MOSFETs carry the whole current for the two electrical speed controllers. The external physical switches directly control the voltage on the gates. For safety reasons, no software logic is behind the power control system.

MOSFETs, compared with regular mechanical relays, have huge advantages over the whole life cycle [18]. However, due to the characteristics of the PN junction, we need to do some calculations to ensure that, under full workload, the thermal rise of the MOSFETs is within the operational range.

Table 3.2: Technical Specifications of TOSHIBA TK4R3E06PL (part)

Characteristics	**Symbol**	**Rating**
Maximum drain-source voltage	V_{DSS}	60 V
Maximum gate-source voltage	V_{GSS}	20 V
Maximum drain current (DC)	I_D	80 A
Maximum channel temperature	T_{ch}	175 °C
Channel-to-case thermal resistance (T=25 °C)	$R_{th(ch-c)}$	1.72 °C/W
Drain-source on-resistance at $V_{GS} = 10\ V, I_D = 34\ A$	$R_{DS(ON)}$	3.3mΩ (Typical)

3.1.2 Thermal Estimation

Because one MOSFET controls only one thruster, the maximum load for each MOSFET approximately equals the full power of the thruster. We ignored the other power consumers like Pi Pico and ESCs since their power ratings are only a few milliwatts. The heat dissipation of the MOSFETs under the maximum workload (24 A), according to Table 3.2, can be calculated through Ohm's Law:

$$P_w = I^2 R_{ds(on)} = 24\ A^2 \times 3.3\ \text{m}\Omega = 1.9008\ W \tag{3.1}$$

Given the room temperature 25°C, the maximum junction temperature under full workload is:

$$T_j = T_r + P_w R_{th(ch-c)} = 25°C + 1.9008\ W \times 1.72C°\ C/W = 28.269376°C < T_{max} = 175\ °C \tag{3.2}$$

Which is well within the temperature range of TK4R3E06PL shown in Table 3.2.

3.1.3 Switch system

3.1.4 Voltage Measuring Point

The central control unit can measure the voltage of the battery system through the exposed ADC port on the Piranha's power board. The voltage on the ADC port is divided from the terminal voltage from the battery system by a series of resistors shown in Figure 3.6.

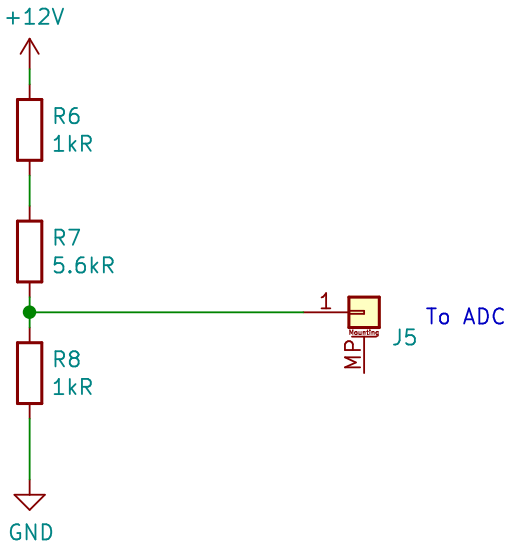

Figure 3.6: The voltage divider circuit. J5 is supposed to connect to the analog input pin of the MCU.

One can calculate the voltage ratio by:

$$H_{ADC} = \frac{R8}{R6 + R7 + R8} = \frac{1}{7.6} \tag{3.3}$$

3.2 Battery System

Piranha uses two paralleled connected Lithium-ion battery modules from BlueRobotics Inc. Each of them comprises 24 SAMSUNG INR18650-30Q cells, combined in a 4S6P configuration, shown in the Figure 3.7. Table 3.3 lists the technical specification of a single module.

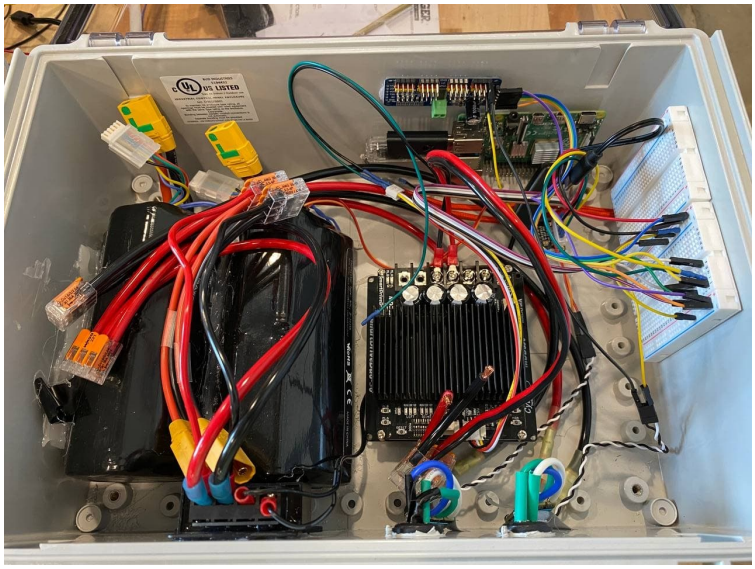

Figure 3.7: I took the picture at an early stage test. The battery packs on the left are glued to the box to prevent collisions.

Table 3.3: Technical Specifications of BlueRobotics Lithium-ion Battery Module (14.8 V, 18 Ah)

Characteristics	**Rating**
Nominal Voltage	14.8 V
Nominal Capacity	18.0 Ah
Cell Configuration	$4S6P$
Maximum Continuous Current Draw	90 A

3.3 Motors (BLDC)

Piranha equipped two BlueRobotics T200 Thrusters at the rear, shown in Figure 3.8. By the aerospace terminology convention, these thrusters are called motors in this article. They generate the main propulsion force for the vehicle.

Table 3.4 lists the technical specifications of the motor.

Figure 3.8: I calibrated two thrusters with a game controller. The right thruster is spinning. Meanwhile, the left one is not moving.

Table 3.4: Technical Specifications of BlueRobotics T200 Thruster (part)

Characteristics	**Rating**
Full Throttle FWD/REV Thrust Nominal (16 V)	5.25 / 4.1 kgf
Operating Voltage	$7 - 20$ V
Full Throttle Current Nominal (16 V)	24 A
Full Throttle Power Nominal (16 V)	390 W

3.4 Electric Speed Controller (ESC)

The speed controller model is BlueRobotics Basic ESC, shown in Figure 3.9. The ESCs on Piranha are controlled by two pulse-width modulated signals (PWM) generated by Pi Pico.

Table 3.5 lists their technical specifications.

3.5 MCU

The microcontroller unit, also known as MCU, is the Pi Pico, designed and manufactured by Pi Foundation, shown in Figure 3.10. Its specification can be found in Table 3.6.

The chip model on the Pi Pico board is RP2040. It has a dual-core cortex M0+ at up to 133MHz

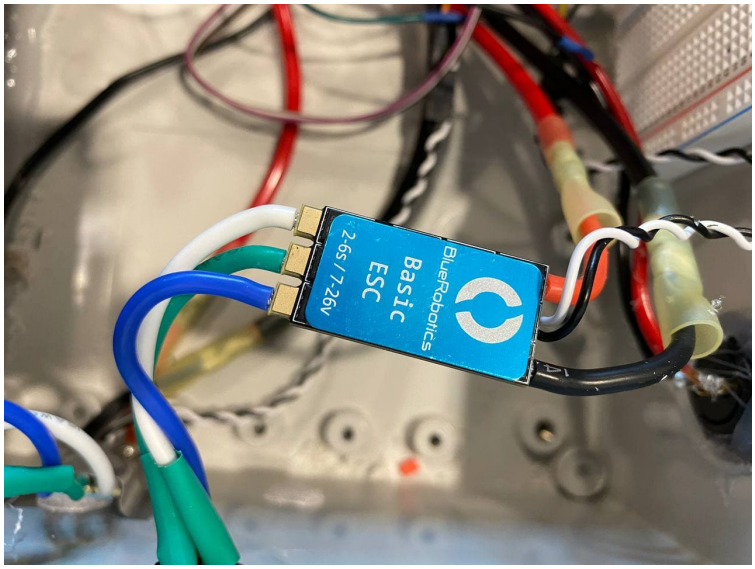

Figure 3.9: BlueRobotics Basic ESC.

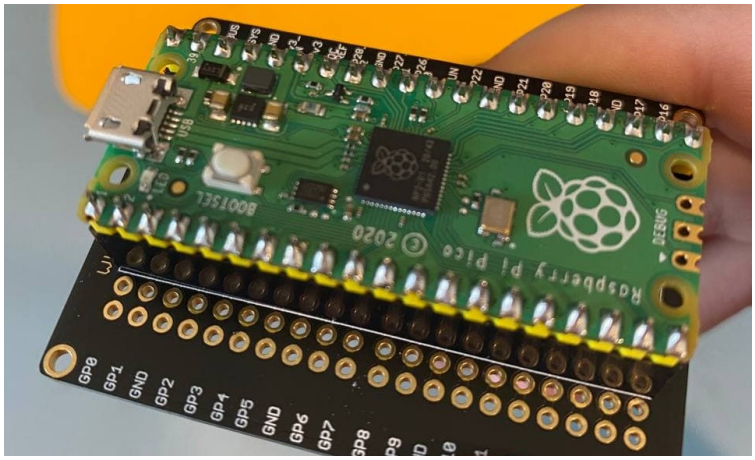

Figure 3.10: Pi Pico installed on the breakout board.

Table 3.5: Technical Specifications of BlueRobotics Basic ESC (part)

Characteristics	**Rating**
Operating Voltage	$7 - 26\ V$
Maximum Current	$30\ A$
Maximum Update Rate	400 Hz
Stop Signal Width	$1500\ \mu s$
Maximum Forward Signal Width	$1900\ \mu s$
Maximum Backward Signal Width	$1100\ \mu s$

frequency. For its specification, please refer to the extra document files.

Table 3.6: Electrical Specifications of Pi Pico (part)

Characteristics	**Rating**
Supply Voltage	$1.8 - 5.5V$
Operating Temperature	$-20 - 85\ ^{\circ}C$
VBUS Current	$1.2 - 92.8\ mA$
Flash Size	2 MByte

3.6 IMU

On Piranha, there are two IMUs. One is Adafruit TDK InvenSense ICM-20948 9-DoF shown as in Figure 3.11, the other is Ceva BNO085 IMU, shown in Figure 3.12.

ICM-20948 is the latest 9-axis MotionTracking device designed by TDK. It contains a 3-axis MEMS accelerometer, a 3-axis MEMS gyroscope, and a 3-axis MEMS magnetometer. It also has a built-in temperature sensor that can compensate for the error caused by the temperature shifting. The specification of ICM-20948 can be found in Table 3.7. ICM20948 features a built-in digital motion processor to process the acceleration data on the chip, so the Pi Pico can directly access

the real-time position and speed data.

BNO085 is co-designed by Ceva and Bosch. Like ICM20948, it also integrates a triaxial accelerometer, a triaxial gyroscope, a magnetometer, and a 32-bit ARM® Cortex™-M0+ microcontroller running CEVA's SH-2 firmware that can output the integrated position result directly. Its specification can be found in Table 3.8.

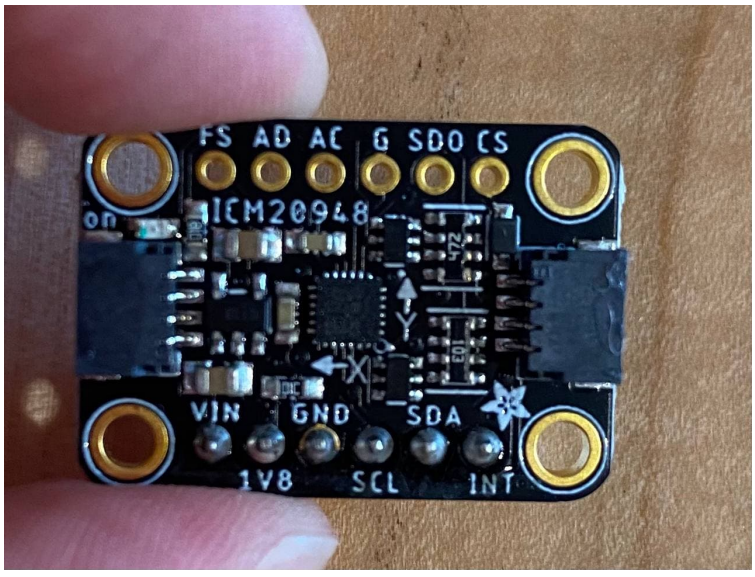

Figure 3.11: Adafruit TDK InvenSense ICM-20948 9-DoF IMU.

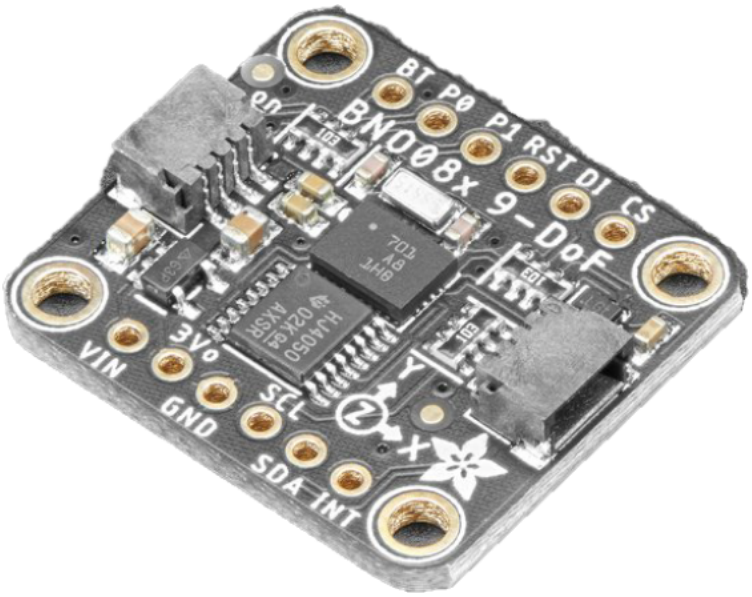

Figure 3.12: Adafruit 9-DOF Orientation IMU Fusion Breakout - BNO085.

Table 3.7: Specification of ICM-20948 (part)

Characteristics	**Rating**
Gyroscope Date Rate (Low Pass Filter On)	1.125 kHz
Accelerometer Date Rate (Low Pass Filter On)	1.125 kHz
Magnetometer Date Rate	100 Hz
Gyroscope Full-Scale Range (GYRO_FS_SEL=0)	± 250 *dps*
Accelerometer Full-Scale Range (ACCEL_FS=0)	± 2 G
Magnetometer Full-Scale Range	± 4900 μT

Table 3.8: Specification of BNO-085 (part)

Characteristics	**Rating**
Gyroscope Date Rate (Low Pass Filter On)	1 kHz
Accelerometer Date Rate (Low Pass Filter On)	0.5 kHz
Magnetometer Date Rate	100 Hz
Gyroscope Full-Scale Range (GYRO_FS_SEL=0)	± 2000 *dps*
Accelerometer Full-Scale Range (ACCEL_FS=0)	± 8 *G*

3.7 Magnetometer

Since electromagnetic interference can easily disrupt magnetometer readings, Piranha also has a dedicated magnetometer, RM3100, installed away from the motors, batteries, and other electrical components. The RM3100 sensor is shown in Figure 3.13, its technical specification is listed in Table 3.9.

Figure 3.13: WitMotion High-Precision RM3100 Magnetometer Sensor

One must verify that sensitivity of the magnetometer measurement is high enough to detect

Table 3.9: Specifications of RM3100 (part)

Characteristics	**Rating**
Maximum Single-Axis Sample Rate	1600/850/440 Hz
DC Supply Voltage	$2.0 \sim 3.6$ V
Field Measurement Range	$-800 \sim 800$ $m\mu$T
Sensitivity	50/26/13 nT
Noise	30/20/15

the earth's magnetic field, which is approximately 0.6 Gauss (48A/m; 6000 nT), before using it. RM3100 has a measurement range within ± 800 μT or $\pm 8 \times 10^5$ nT, which is well more significant than the earth's magnetic field strength, and high sensitivity as low as ± 13 nT which is only 0.21% of the earth's magnetic field strength. So it is reasonable to use RM3100 as a digital compass on Piranha.

In applications like small fixed-wing planes and rotors, the engineers often consider magnetometer readings very noisy because the electromagnetic interference is usually very high. However, on Piranha, because the magnetometer is put far from the interference source, we believe the data to be reliable and put more weight on them in the sensor fusion (EKF) part. This report will explain it later.

3.8 GPS

GPS is essential in the position estimation process because it provides drift-free position and speed data in real-time.

On Piranha, the GPS module is Adafruit Ultimate GPS, built around the MTK3339 chipset. Readers can find its technical specifications in Table 3.10.

Table 3.10: Specification of Adafruit Ultimate GPS (part)

Characteristics	**Rating**
Satellites	22 tracking, 66 searching
Update Rate	$1 - 10$Hz
Position Accuracy	$< 3\ m$
Velocity Accuracy	$< 0.1\ m/s$
Maximum Velocity	$515\ m/s$

3.9 Wireless Module

We designed Piranha with a manual mode, which allows the human operator to control the boat from a distance. During the development, we considered multiple different communication methods and tested them all. Table 3.11 shows the comparison among them.

Table 3.11: Wireless Communication Method Comparison

Method	**Speed**	**Range**	**Cost**	**Power Consumption**
4G/LTE	100 Mbps	Depends on the coverage	High[1]	100 mW
WiFi	150 Mbps	$<$ 70 ft	Low	100 mW
RF Radio	10 Mbps	$\sim$ 1 mile	High	$\sim$ 1 W
LoRa	$0.3 - 20$kbps	$\sim$ 5 miles	Low	$\sim$ 1 W

Piranha is a water surface robot, so its moving range depends on the control signal range if controlled by a human operator. For this reason, the control signal range should be as long as possible. Since the control signal only has simple commands, it does not need high bandwidth to work.

As a result, We chose to use LoRa on Piranha as the main wireless communication method from the ground station. LoRa is a wireless communication protocol based on the spread spectrum

[1]Most 4G plans in the U.S. need a monthly payment.

Figure 3.14: E32-915T30D

technology. It is designed for low power, and long-range data transmission [11]. The LoRa module on Piranha is the Ebyte E32-915T30D model, shown in Figure 3.14, it works on the 915MHz band. Because it works on an unlicensed band in the U.S., there is no need to get a commercial license from FCC. Its technical specification can be found in Table 3.12.

Table 3.12: Specifications of E32-915T30D

Characteristics	**Rating**
Maximum Transmission Power	1 W
Maximum Communication Distance	8 km
Air Date Rate	$0.3 - 19.2$ kbps
Maximum Tx Power	30 dBm
Maximum Rx Power	-147 dBm

Chapter 4

Simulation

Simulation is an essential step in the control system design. The simulation model should reflect the dynamics property of the controlled target as close as possible. With a simulated model, engineers can first design and test the controller in the virtual environment and then deploy them in the real world.

The advantage of a simulation-based development is that it will not cause any damage to the original model, which is usually very expensive and hard to build. Second, engineers can obtain the results in the computer faster than in the real world, which means we can get the results of a simulated test way more quickly than a real field test.

This chapter is about the simulation of Piranha. The formulation is based on a 6-DoF non-linear rigid body system. Unlike ground vehicles like rovers and cars, a water surface vehicle like Piranha has much more complicated dynamics because of fluid mechanics.

The most common ways to formulate the EOM of a given system are either through Lagrangian mechanics or Newtonian mechanics. Because the simulation only has one rigid body target, we preferred Newton's Law in this application. However, Newton's Law only holds within the inertial frame. Here, some assumptions are made before the EOM formulation to ensure this condition is

met [10].

4.1 Assumptions

Before the formulation, we made a few assumptions to rule out some unnecessary factors to consider, such as the Coriolis force and the earth's curvature. The assumptions include:

1. The water surface is calm and flat without any curves.
2. The vehicle is a rigid body with a constant mass.
3. The gravitational acceleration is also a constant value.
4. The simulated world does not rotate.
5. The fluid has a constant current direction and speed, as well as the wind.

These assumptions make sure that the simulation world frame is inertial. In other words, we can use Newton's Second Law to formulate the equations of motion.

4.2 6 DoF State-Space Equations of Motion

The equation of motion has 6 degrees of freedom. To simplify the formulation, the state vector $\boldsymbol{X}$ is divided into two parts, $\boldsymbol{x}$ and $\boldsymbol{\nu}$.

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x} \\ \\ \boldsymbol{\nu} \end{bmatrix} \tag{4.1}$$

We define $\boldsymbol{x}$ as the position vector, $\boldsymbol{\nu}$ as the velocity vector. They are attached to different reference frames, $\boldsymbol{x}$ is attached to the ground NED frame while $\boldsymbol{\nu}$ is linked to Piranha's body frame.

$$\boldsymbol{x} = [p_n, p_e, p_d, \phi, \theta, \psi]^\top \tag{4.2}$$

$$\boldsymbol{v} = [u, v, w, p, q, r]^\top \tag{4.3}$$

The input vector of the system $\boldsymbol{u}_{\text{PWM}}$ consists of the PWM input values for the left and right thrusters.

We can write the EOM of this system as:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{C}(\boldsymbol{x}, \boldsymbol{\nu}, \boldsymbol{w}_c) + \boldsymbol{D}(\boldsymbol{x}, \boldsymbol{\nu}, \boldsymbol{w}_a) + \boldsymbol{G} = \boldsymbol{\tau} \tag{4.4}$$

$\boldsymbol{M}$ is the mass matrix, $\boldsymbol{C}(\boldsymbol{x}, \boldsymbol{\nu}, \boldsymbol{w}_c)$ is the fluid dynamical matrix, $\boldsymbol{w}_c$ is the water current vector, $\boldsymbol{D}(\boldsymbol{x}, \boldsymbol{\nu}, \boldsymbol{w}_a)$ is the air dynamical matrix, $\boldsymbol{w}_a$ is the wind speed vector, $\boldsymbol{G}$ is the gravity matrix, $\boldsymbol{\tau}(\boldsymbol{u}_{\text{PWM}})$ is the mapped input vector of the system, the transformation will be discussed in after a few sections.

4.3 Reference Frames

In the state-space representation shown above, we represent all components as vectors. However, these vectors in the simulation EOM link to different frames, such as the input vector $\boldsymbol{\tau}(\boldsymbol{u}_{\text{PWM}})$ is in the body frame. Meanwhile, the gravitational component G is a constant vector in the ground frame.

To begin with, here list five types of frames in the simulation:

- Earth-Centered Inertial (ECI) frame.
- Earth-Centered, Earth-Fixed (ECFF) frame.
- Ground North, East, Down (NED) frame.

- Local North, East, Down frame.
- The Body frame.

Because the assumptions rule out the world's rotation, the Coriolis effect does not exist in the simulation. Because Piranha only moves on the surface of the earth, we consider the $\boldsymbol{x}$ position vector is in the ground NED frame rather than the ECFF frame and ECI frame.

Since we use Newton's Law for building the simulation EOM, every component inside the equation is relative to the body frame. However, some parts like the wind speed vector and the gravity matrix are initially constant in the local NED frame, so we must transform them into the same frame first.

4.3.1 Transform Vectors from Local NED Frame to Body Frame

With the roll, pitch, yaw angles to be $[\phi, \theta, \psi]$, the transform matrix can be described by:

$$R_1 = \begin{bmatrix} \cos(\psi) & \sin(\psi) & 0 \\ -\sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{4.5}$$

$$R_2 = \begin{bmatrix} \cos(\theta) & 0 & -\sin(\theta) \\ 0 & 1 & 0 \\ \sin(\theta) & 0 & \cos(\theta) \end{bmatrix} \tag{4.6}$$

$$R_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & \sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix} \tag{4.7}$$

$$\boldsymbol{x}_{\text{body}} = R_3 R_2 R_1 \boldsymbol{x}_{\text{ground}} \tag{4.8}$$

4.4 Mass Matrix

First decompose the velocity vector $\boldsymbol{\nu}$ into linear movement $\boldsymbol{\nu}_l = [u,\ v,\ w]$ and rotation parts $\boldsymbol{\nu}_r = [p,\ q,\ r]$. The mass matrix can also be split into the linear mass matrix $\mathbf{M}_l$ and the inertia matrix $\mathbf{M}_r$.

$$\mathbf{M} = \begin{bmatrix} \mathbf{M}_l & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_r \end{bmatrix} \tag{4.9}$$

Where,

$$\mathbf{M}_l = m\mathbf{I} \quad \mathbf{M}_r = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \tag{4.10}$$

And,

$$\mathbf{F} = \mathbf{F}_0 + \Delta\mathbf{F} \tag{4.11}$$

$$\mathbf{T} = \mathbf{T}_0 + \Delta\mathbf{T} \tag{4.12}$$

$\mathbf{F}_0$ and $\mathbf{T}$ is the theoretical force and torque. The Δ denotes the perturbation in the real-world model. In this section, we ignore this perturbation.

With Newton's Law,

$$\mathbf{F} = \begin{bmatrix} m\dot{u} & \cdots & 0 \\ \vdots & m\dot{v} & \vdots \\ 0 & \cdots & m\dot{w} \end{bmatrix} \quad \mathbf{T} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} \tag{4.13}$$

$$\mathbf{M}\dot{\nu} = \begin{bmatrix} \mathbf{F} & \mathbf{0} \\ \mathbf{0} & \mathbf{T} \end{bmatrix} \tag{4.14}$$

In the simulation, these values are obtained from SOLIDWORKS Mass Properties, shown in Figure 4.1:

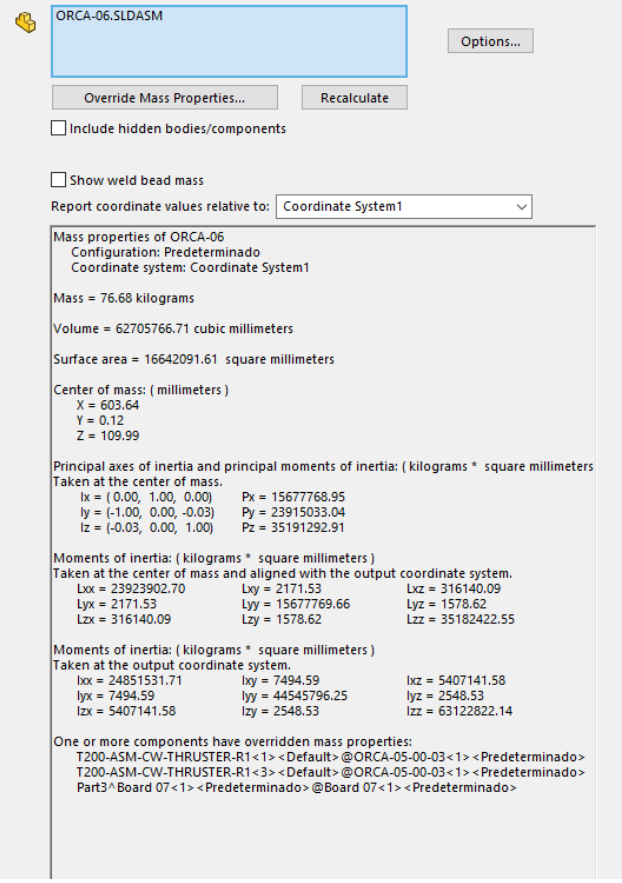

Figure 4.1: The mass matrix values of Piranha generated in the SOLIDWORKS Mass Properties menu.

4.5 Fluid Dynamical Model

The fluid dynamical model includes two parts, hydrostatic and hydrodynamic parts.

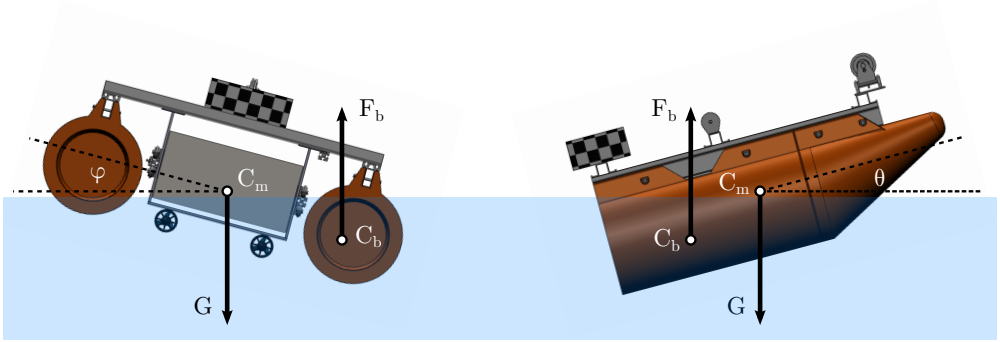

Figure 4.2: Hydrostatic model of Piranha.

4.5.1 Hydrostatic analysis

In the hydrostatic part, the goal of the simulator is to estimate the whole buoyant force applied to Piranha and the buoyancy center, given the position of the boat. In Figure 4.2, we can see that the buoyancy estimation includes the center of buoyancy estimation and the upward force estimation. According to Archimedes' principle, we can calculate the upward buoyant force through the volume of the displaced water.

We can get estimations with pure mathematical calculation. For simplicity, here we do the math in the body frame. The first step is to transform the water level back to the Piranha's body frame, shown in the following equation.

$$F_{wg}(x, y) = 0 \longrightarrow F_{wb}, \quad F_{wg},\ F_{wb} \in R^2 \tag{4.15}$$

We can consider the water level surface as a plane under the assumptions. So the equation of a plane in 3D space is:

$$ax + by + cz = d \tag{4.16}$$

The normal vector can be represented as $\boldsymbol{n_1} = [a, b, c]^\top$. We can determine the water surface in the body frame by solving this equation:

$$R_v^b \boldsymbol{n_1} \cdot \boldsymbol{x}_{(x=0,y=0)}^{\top} = p_d \tag{4.17}$$

There are three cases in calculating integral:

1. The water level surface only intersects the lower half part of the float.

2. The water level surface only intersects the upper half part of the float.

3. The water level surface intersects both the upper and lower halves of the float.

For all three cases, the integral can be represented as the following equation, denote F_s to be the separate plane:

$$V = \iint_{A_{xy}} [\min(\max(F_w - F_s, 0), F_u - F_s) + \min(\max(F_w - F_l, 0), F_s - F_l)] \tag{4.18}$$

Because it is not a differentiable function due to *min* and *max* operators, but it can be solved using modern numerical methods. The formulation is not exclusive. These variables inside the integrated function can be (x, z) instead of (x, y). However, the function is always not differentiable.

The buoyancy center is the mass center of the displaced water, thus can be calculated by the center of mass formula:

$$M_{yz} = \iiint x\rho(x, y, z) \, \mathrm{d}V \tag{4.19}$$

$$M_{xz} = \iiint y\rho(x, y, z) \, \mathrm{d}V \tag{4.20}$$

$$M_{xy} = \iiint z\rho(x, y, z) \, \mathrm{d}V \tag{4.21}$$

$$\overline{x} = \frac{M_{yz}}{m}, \ \overline{y} = \frac{M_{xz}}{m}, \ \overline{z} = \frac{M_{xy}}{m} \tag{4.22}$$

4.5.2 Hydrodynamic analysis

The hydrodynamic analysis calculates the parasitic forces in the simulation. This process in real-life applications is usually handled by using modern CFD simulators to solve the Navier-Stokes equations.

Although the CFD simulation precision depends on the fineness of the mesh, limited by today's computer technology, the CFD simulation is not real-time. Usually, it takes a long time, ranging from a few minutes to hours, to calculate for a set of given parameters. However, the CFD simulation needs to run from tens to thousands of times per second in the simulation scenarios, depending on the time step. As a result, the time cost for calculating hydrodynamics using common CFD software is unacceptable in the simulation.

The following section introduces an algorithm, which is used in Piranha's simulation code. This algorithm takes a triangle mesh as the input model and does the hydrostatic and hydrodynamic estimation at the cell level first. After that, it sums up all results and produces a general estimate of the buoyancy and drag on the whole body of the boat.

4.6 Numerical Calculation

This section covers the numerical methods used in Piranha's simulation program. We cannot calculate the theoretical equation efficiently in the computer. As a result, a simplified mesh model of Piranha is used to accelerate the calculation.

4.6.1 Mesh Simplification

The first step is to simplify the mesh and reduce the number of triangles to reduce the calculation time cost. The aim is to use the fewest triangles but maintain the shape of the original model.

We represent Piranha's simulation model with two cylinders and two cones, shown in Figure

4.3.

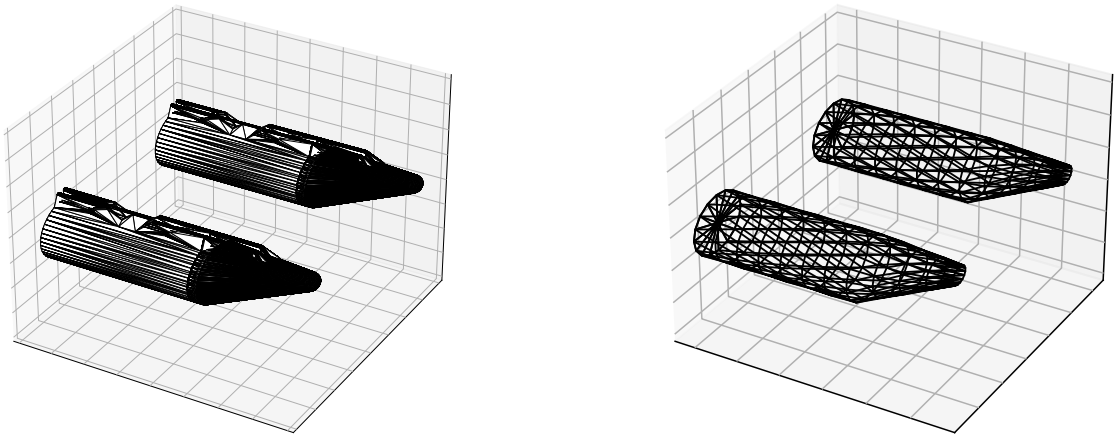

Figure 4.3: On the left is the original mesh generated by SOLIDWORKS, on the right is the simplified mesh model.

Buoyancy Estimate

The key to estimating buoyancy is to calculate the submerged volume. For a single triangle in the mesh, we can calculate its volume by the surface integral over the projected area, shown in the following equations.

$$\boldsymbol{V} = \int_S F(x,y)\ \mathrm{d}x\mathrm{d}y \tag{4.23}$$

$$F(x,y) = \begin{cases} f(x,y), f(x,y) > 0 \\ \\ 0, f(x,y) \leq 0 \end{cases} \tag{4.24}$$

Given the triangle plane function $z = f(x,y)$.

If we ignore every triangle that is not fully submerged, there is also another way to calculate the volume under a triangle without using integration.

If we have a 3D triangle defined by (x_1, y_1, z_1), (x_2, y_2, z_2), (x_3, y_3, z_3). Assume they are arranged in the order that $z_1 < z_2 < z_3$.

Assume there is a horizontal plane P, in other words, a plane that is parallel to XY plane crossing the middle point (x_2, y_2, z_3), P divides the 3D triangle into upper and lower parts like Figure 4.4.

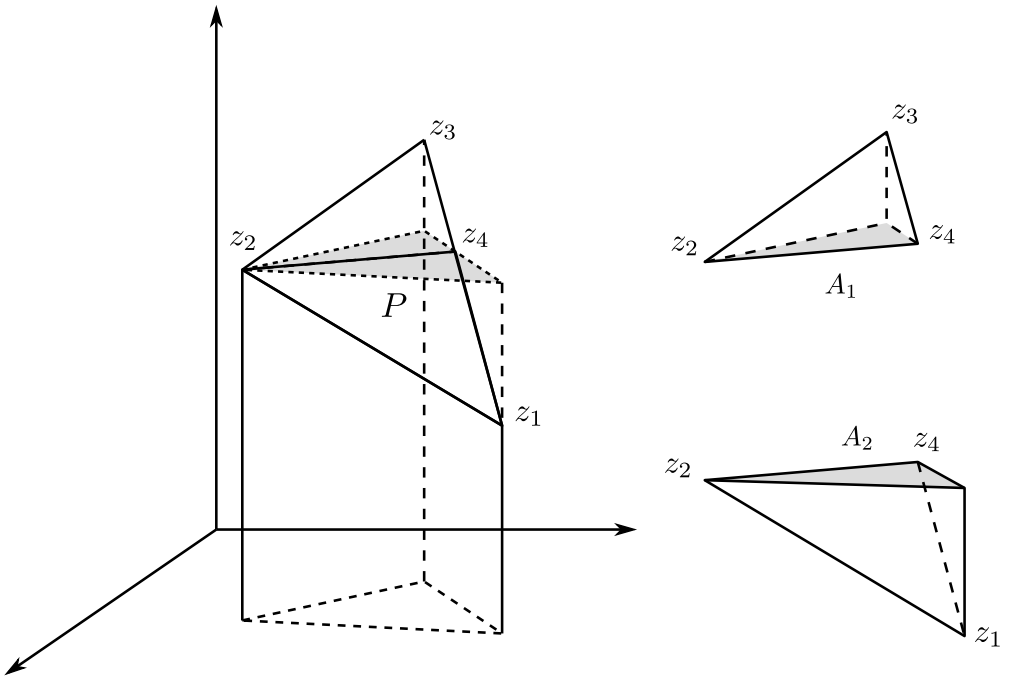

Figure 4.4: A_1 and A_2 are the projected area of the upper part and the lower part on the plane P.

We can calculate the volume of the 3D triangle given above as the volume of the triangular prism made by P, plus the upper triangular cone, minus the lower triangular cone, equals to:

$$V = (A_1 + A_2)z_2 + \frac{1}{3}A_1(z_3 - z_2) - \frac{1}{3}A_2(z_2 - z_1) \tag{4.25}$$

The projected area A_1 and A_2 can be calculated by:

$$A_1 = \frac{1}{2}[(x_4y_2 - x_2y_4) + (x_3y_4 - x_4y_3) + (x_2y_3 - x_3y_2)] \tag{4.26}$$

$$A_2 = \frac{1}{2}[(x_1y_2 - x_2y_1) + (x_4y_1 - x_1y_4) + (x_2y_4 - x_4y_2)] \tag{4.27}$$

Where (x_4, y_4, z_4) is the intersection point between (x_3, y_3, z_3) and (x_1, y_1, z_1). Apparently because (x_4, y_4, z_4) is on P, $z_4 = z_2$. x_4 and y_4 are given by:

$$x_4 = x_1 + ((z_2 - z_1)/(z_3 - z_1))(x_3 - x_1) \tag{4.28}$$

$$y_4 = y_1 + ((z_2 - z_1)/(z_3 - z_1))(y_3 - y_1) \tag{4.29}$$

Substitute back and simplify the equation. In the end, the volume under a 3D triangle can be calculated by:

$$V_{tri} = \frac{1}{6}(z_1 + z_2 + z_3)(A_1 + A_2) = \frac{1}{6}(z_1 + z_2 + z_3)A_{proj} \tag{4.30}$$

The volume equation can be interpreted as the "average height" times the projected area times 1/2.

We can calculate the projected area through the cross product:

$$A_{proj} = \frac{1}{2}|\boldsymbol{u} \times \boldsymbol{v}| \tag{4.31}$$

There is no need for the computational heavy integration step when calculating the volume with this volume formula. However, we can only use it when the triangle is fully submerged.

However, if a triangle is not fully submerged, it is either partially submerged or not submerged at all. For the latter case, we can ignore the triangle. For the partially submerged case, there are two chances:

1. Only one vertex is in the water.

2. Two vertices are in the water.

The simulator takes extra steps for these two cases, shown in Figure 4.5. The key is to find the two intersection points and calculate the cone volume.

$$\text{For case 1:} \quad V = V_{cone} \tag{4.32}$$

$$\text{For case 2:} \quad V = V_{tri} + V_{cone} \tag{4.33}$$

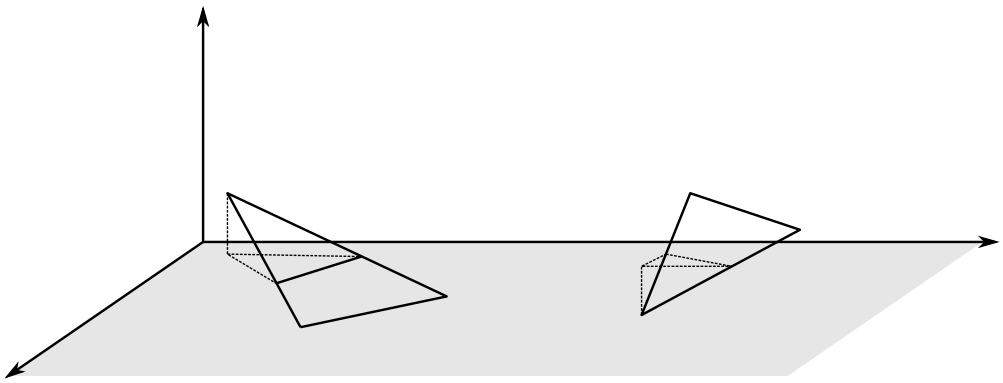

Figure 4.5: Two partially submerged cases.

Given the volume formula, it is easy to calculate the underwater volume for one triangular cell. The next step is to repeat this step for each triangle and sum up the results.

The face direction determines the volume should be taken from the sum, or added to the result, shown in Figure 4.6.

During the calculation, the program determines the face direction by the normal vector of the triangle. Due to this fact, the simulator saves the normal vectors and the triangle mesh for further calculation. These normal vectors, if visualized, can be seen in Figure 4.7:

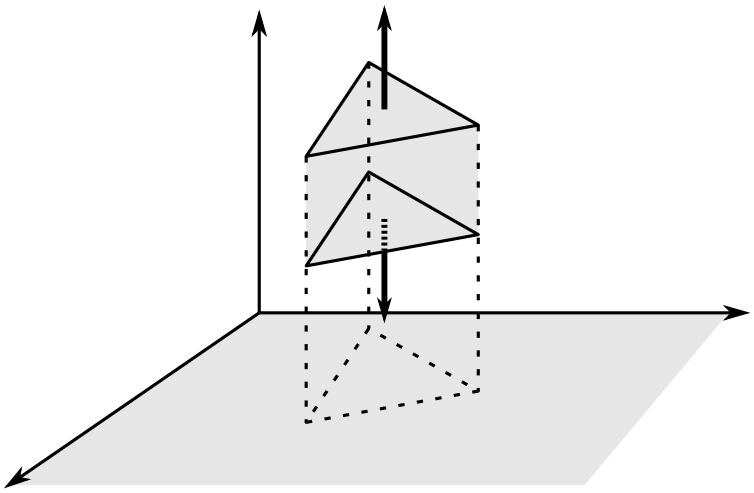

Figure 4.6: The offset water volume is between the two triangles. The face directions for the two triangles are different.

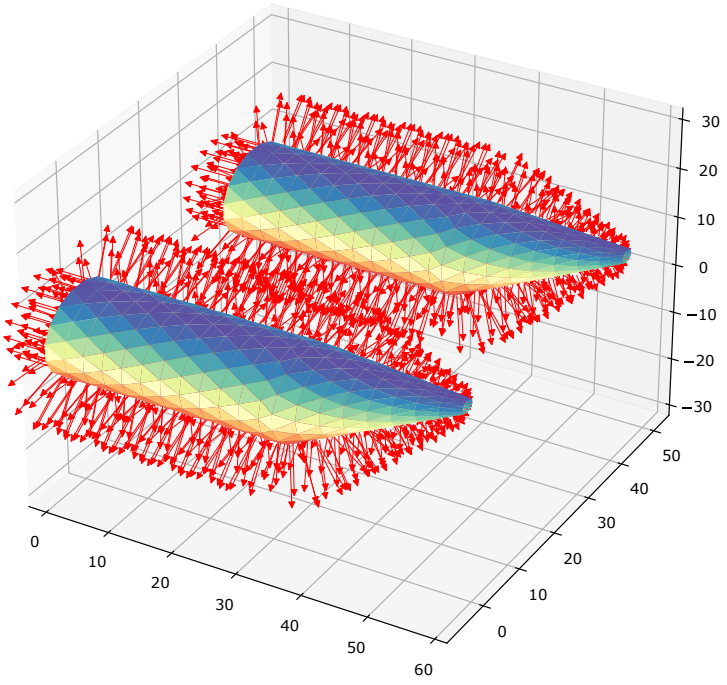

Figure 4.7: In the mesh model, the area of a triangular face equals the length of its normal vector.

Figure 4.8 briefly illustrates the flowchart of the buoyancy estimate algorithm. However, in practice, the simulator does not follow this chart strictly. In the real world, the simulator combines

the hydrostatic estimator and hydrodynamic estimator so that the face classification process only needs to run once for each face. The classification result is shared between both estimators for performance optimization.

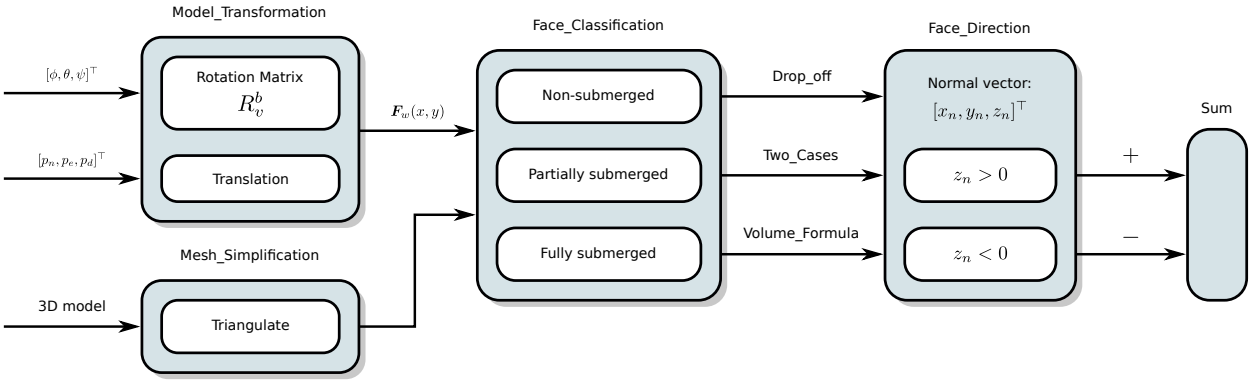

Figure 4.8: Piranha's hydrostatic estimator flowchart.

Drag & Skin Friction Estimate

For simplicity, we only consider drag and skin friction in the simulation's hydrodynamics. The other factor like water waves and twirls are not considered here [17]. We can calculate these two forces through:

$$\begin{aligned} F_d &= \int C_{i0} + C_{i1}\rho v_1 + C_{i2}\rho v_1^2 \mathrm{d}A \\ F_f &= \int C_{f0} + C_{f1}\rho v_2 C_{f2}\rho v_2^2 \mathrm{d}A \end{aligned} \tag{4.34}$$

However, unlike most common simulators, which only use a rough frontal area value in the formula, Piranha's simulator uses a more precise algorithm to calculate the frontal area.

The drag and skin friction are calculated independently for each triangular cell. The simulator sums up all the result values and synthesizes a general force and general torque to represent the hydrodynamic component, shown in Figure 4.9.

The hydrodynamics estimator flowchart can be found in Figure 4.10.

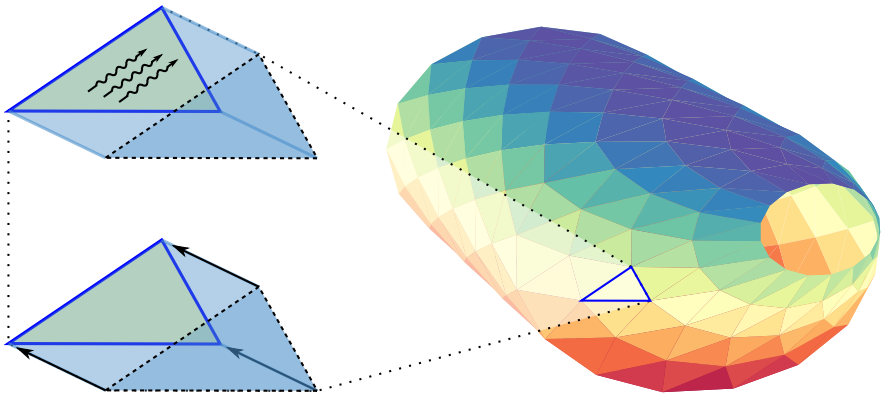

Figure 4.9: Cell level dynamics, drag and skin friction.

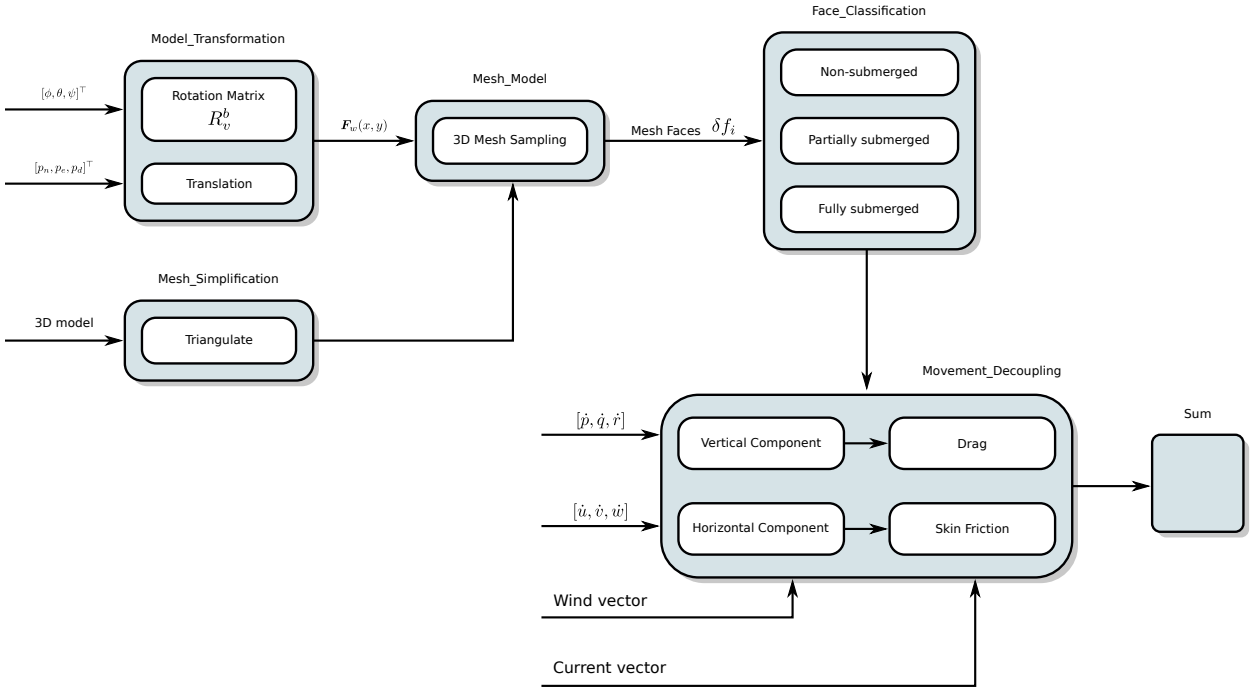

Figure 4.10: Piranha's hydrodynamic estimator flowchart.

The flowchart shows that each face is classified into different types according to the above discussion first. And for simplicity, the simulator ignores the skin friction component for the area in the air.

Wind and Current Vectors

Wind and water current have a significant effect on the simulation. Thus, the simulator cannot ignore them blindly. However, compared with hydrodynamics, the aerodynamics of Piranha is of less importance.

In the simulation, these factors are considered in the hydrodynamics estimation part, as shown in Figure 4.10. While calculating the drag and friction for a face, the simulator first calculates a relative speed for the wind frame and water current frame, then proceeds to the movement decoupling step and applies drag and skin friction coefficients correspondingly.

4.7 Input Mapping

The simulation input is the PWM control signal widths of the two thrusters, which are controllable by the embedded controller. However, the PWM pulse widths need to be transformed into propulsion forces first.

According to the BlueRobotics lab tests results attached to this report in the Appendix section 6.1, the propulsion force depends on two factors, the voltage, and the PWM pulse width. We can see the data in Figure 4.11.

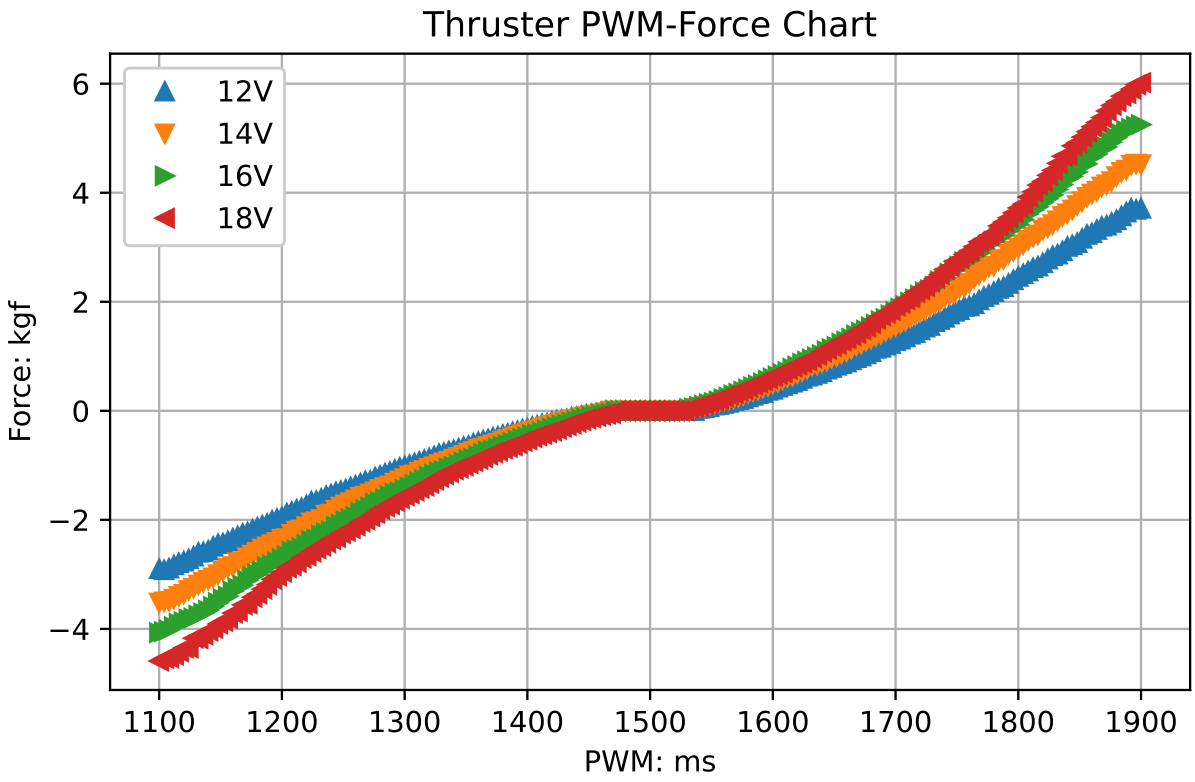

Figure 4.11: BlueRobotics T200 Thruster input-output curve according to the lab test.

The input mapping part in the simulator transforms the input PWM values to the propulsion forces generated by the two thrusters. It works using the nearest point algorithm. At first, it determines which curve to look for the nearest point, then finds the dot with the closest PWM value.

$$\boldsymbol{u}_{\mathrm{f}} = \boldsymbol{F}_{\Delta}(\boldsymbol{u}_{\mathrm{PWM}}) \tag{4.35}$$

The next step is to obtain the torque. According to the geometry definition of the thrusters, the torque can be calculated through the cross-product of the position vector and the propulsion force.

$$\boldsymbol{u}_{\mathrm{q}} = \boldsymbol{v_t} \times \boldsymbol{u}_{\mathrm{f}} \tag{4.36}$$

And at last,

$$\boldsymbol{\tau} = \begin{bmatrix} \boldsymbol{u}_{\mathrm{f}} \\ \\ \boldsymbol{u}_{\mathrm{q}} \end{bmatrix} \tag{4.37}$$

Which is the right hand side of the EOM [16].

4.8 Simulation Results

After building the simulator, we tuned all parameters based on the position and speed data collected during the real-world tests to ensure the simulation was valid. Some parameters are fixed by the mechanical design, such as the gravity center and the mass matrix, so they are determined beforehand.

Other parameters, including the water drag and skin friction coefficients and air drag coefficient (we set the simulator to have a zero skin coefficient for air skin friction), are changeable.

Readers can find three test results in the following few sections.

4.8.1 Drop Test

Drop test is the most straightforward test. In this test, we put Piranha at a certain height above the water at the beginning. When time starts, Piranha starts free-falling until it reaches a stable position where the buoyancy and weight are equal.

Because the input vector is zero all the time, this test can be used to verify the hydrostatic estimator and hydrodynamics estimator work properly. The results are shown in Figure 4.12, 4.13, 4.14, 4.15.

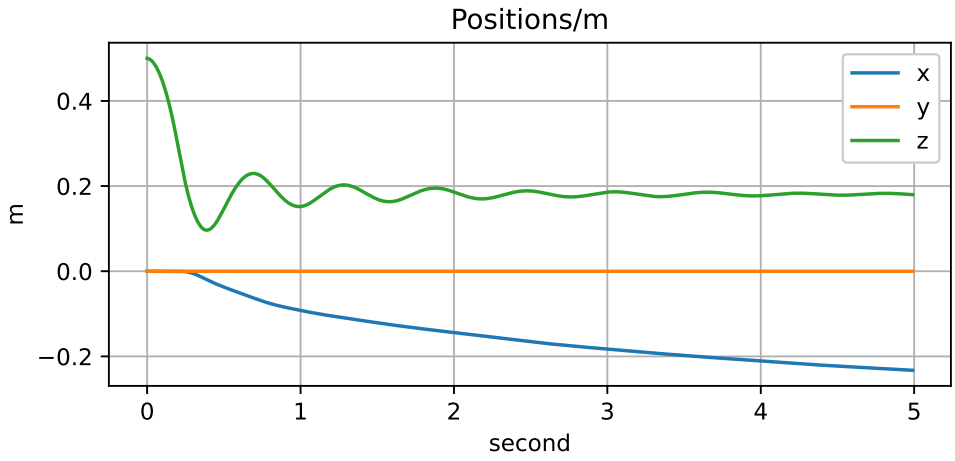

Figure 4.12: Simulation: Drop test result - Positions x, y, z

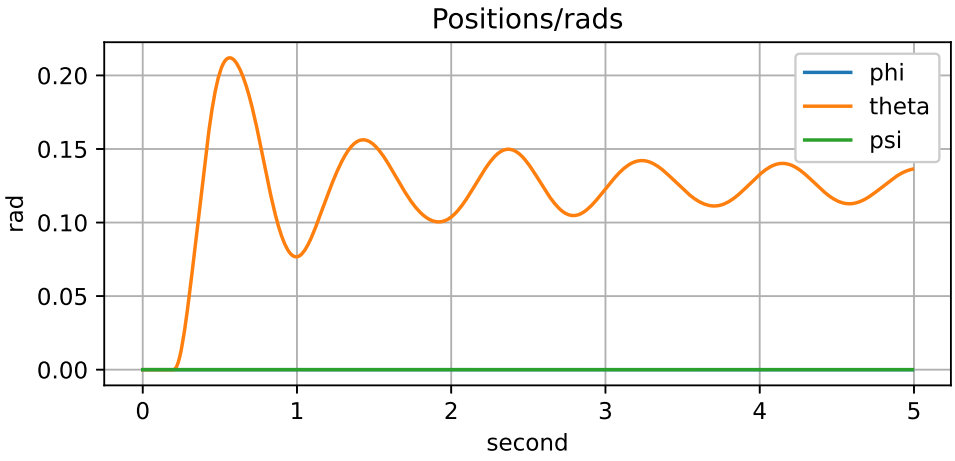

Figure 4.13: Simulation: Drop test result - Attitude ϕ, θ, ψ

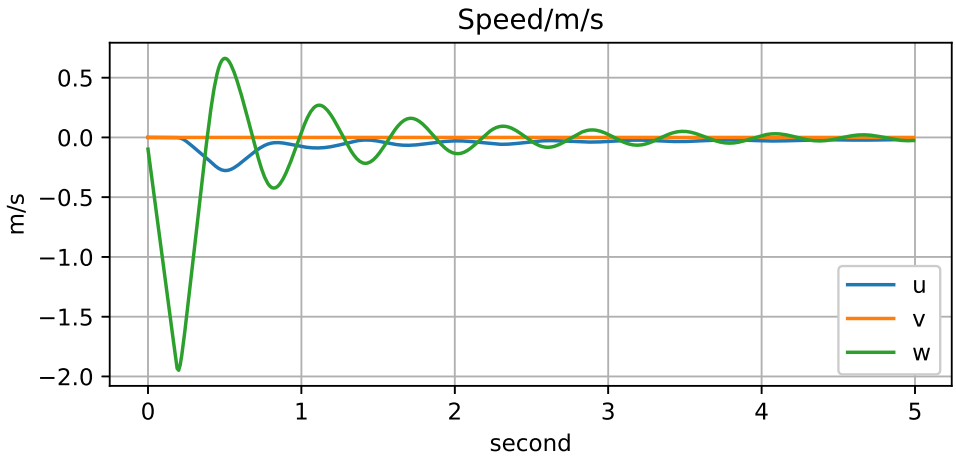

Figure 4.14: Simulation: Drop test result - Linear Speed u, v, w

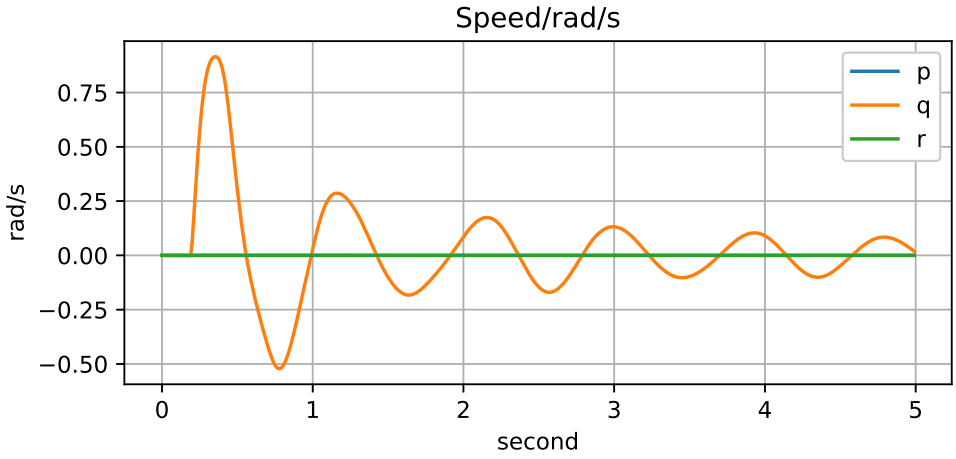

Figure 4.15: Simulation: Drop test result - Angular Speed p, q, r

4.8.2 Linear Driving Test

In the Linear driving test, Piranha starts at a stable state, with a zero speed. When the test starts, the left and right thrusters move forward at maximum input values. The test results are shown in Figure 4.16, 4.17, 4.18, 4.19.

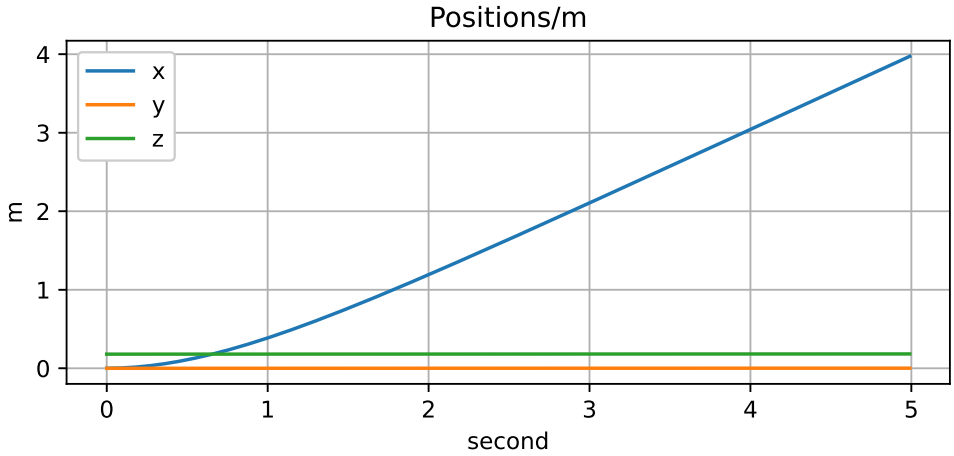

Figure 4.16: Simulation: Linear driving test result - Positions x, y, z

Figure 4.17: Simulation: Linear driving test result - Attitude ϕ, θ, ψ

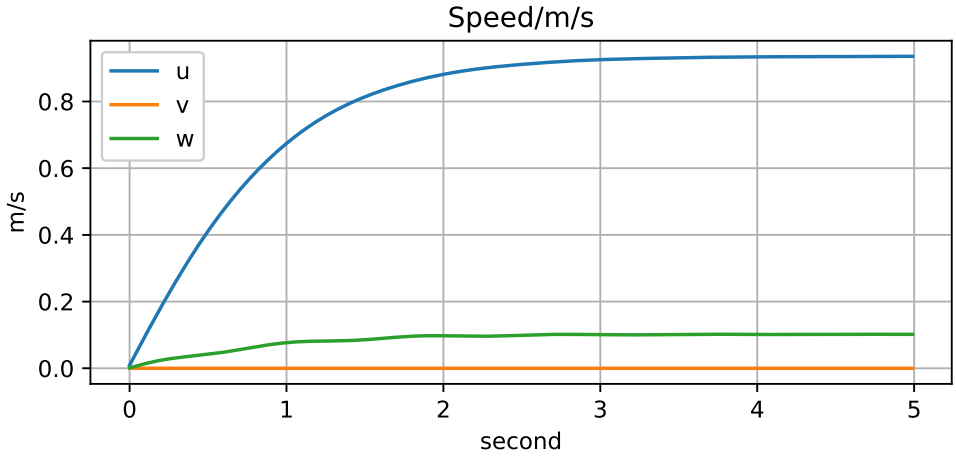

Figure 4.18: Simulation: Linear driving test result - Linear Speed u, v, w

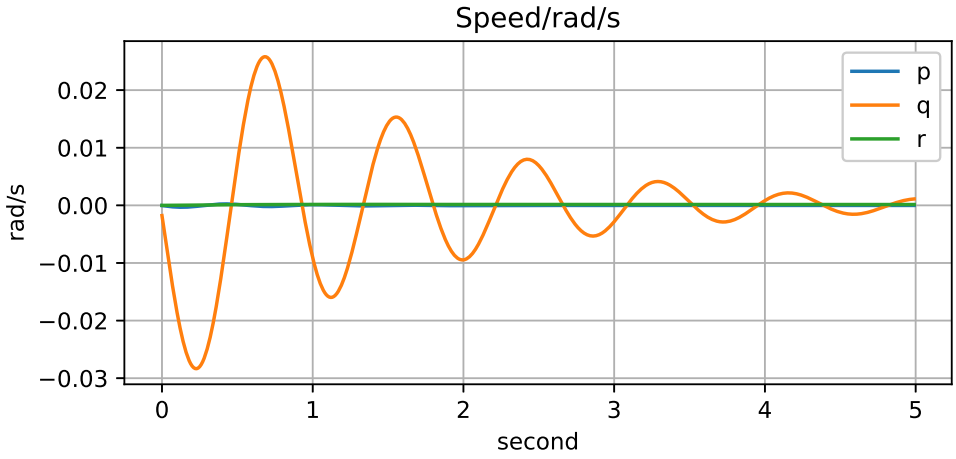

Figure 4.19: Simulation: Linear driving test result - Angular Speed p, q, r

4.8.3 Steering Test

Steering test starts Piranha at the equilibrium point, then the left thruster is given a maximum forward input value, the right thruster is given a maximum backward input value. As a result, the Piranha starts turning. The simulation results can be found in Figure 4.20, 4.21, 4.22, 4.23.

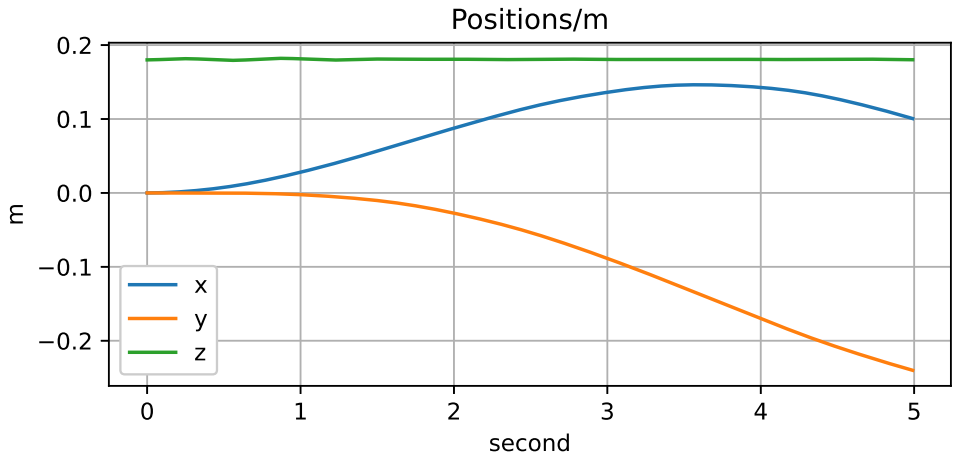

Figure 4.20: Simulation: Steering test result - Positions x, y, z

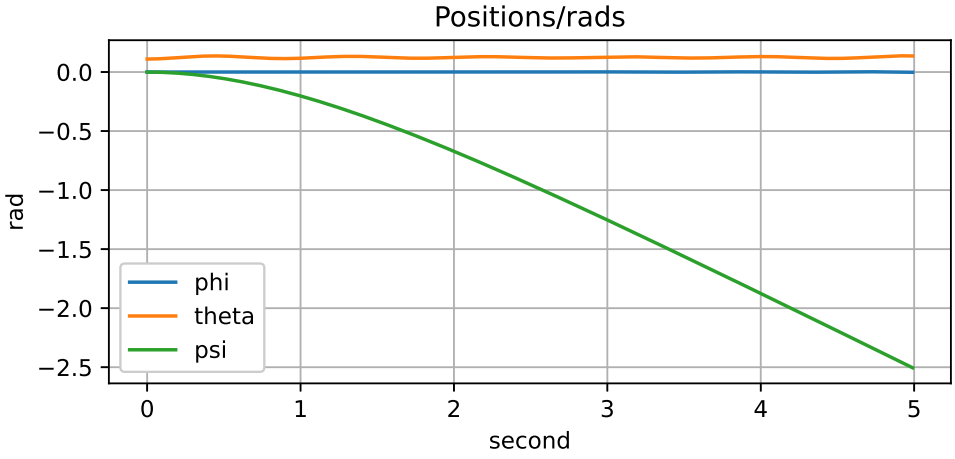

Figure 4.21: Simulation: Steering test result - Attitude ϕ, θ, ψ

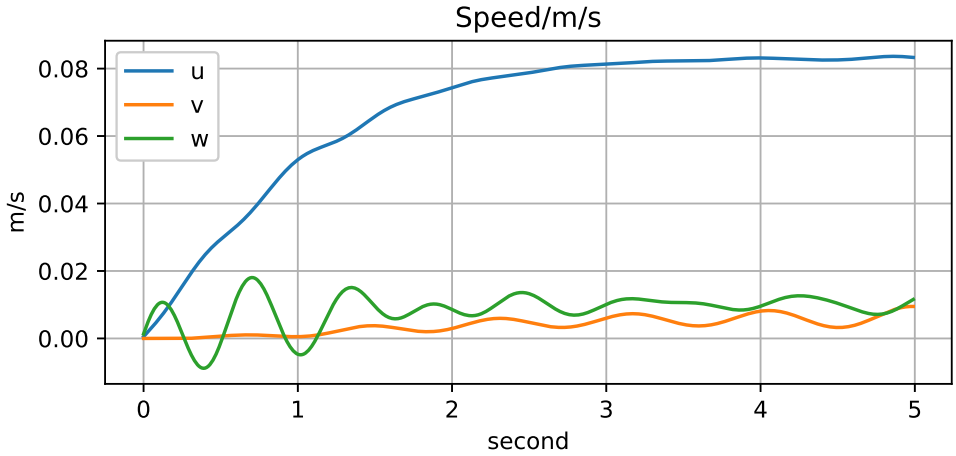

Figure 4.22: Simulation: Steering test result - Linear Speed u, v, w

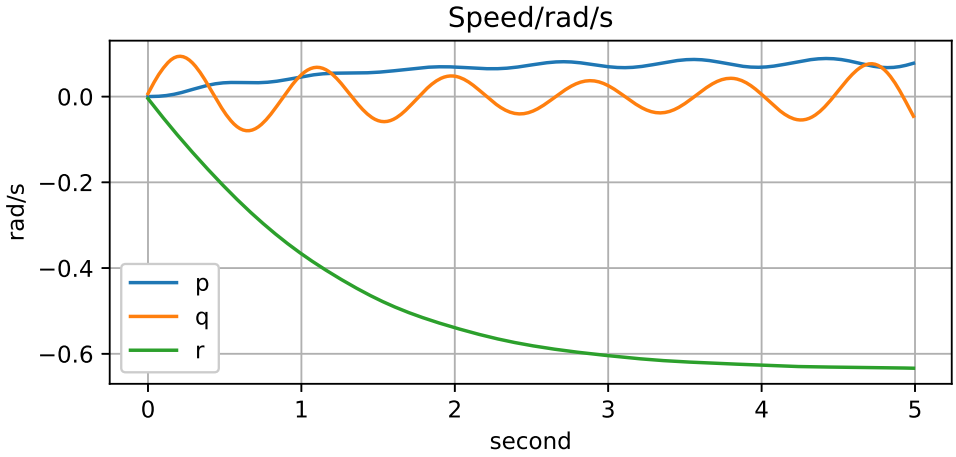

Figure 4.23: Simulation: Steering test result - Angular Speed p, q, r

Chapter 5

Control System

This chapter introduces Piranha's control system design. Unlike the simulation, where the model is 6-DoF and has a 3D environment, we based the control system design on a 3-DoF model representation, also called a navigation model. We did this because Piranha's vertical position is automatically controlled by the fluid physics.

We also ignored the roll, pitch angles. During the field test, the collected data shows the pitch and roll angle of Piranha hardly changed. Readers can verify this fact in the simulation results.

5.1 Navigation Model

The navigation model is a 2D model rather than 3D. This is because the third position value, z, is not of the control system interested [7]. The model frame is shown in Figure 5.1.

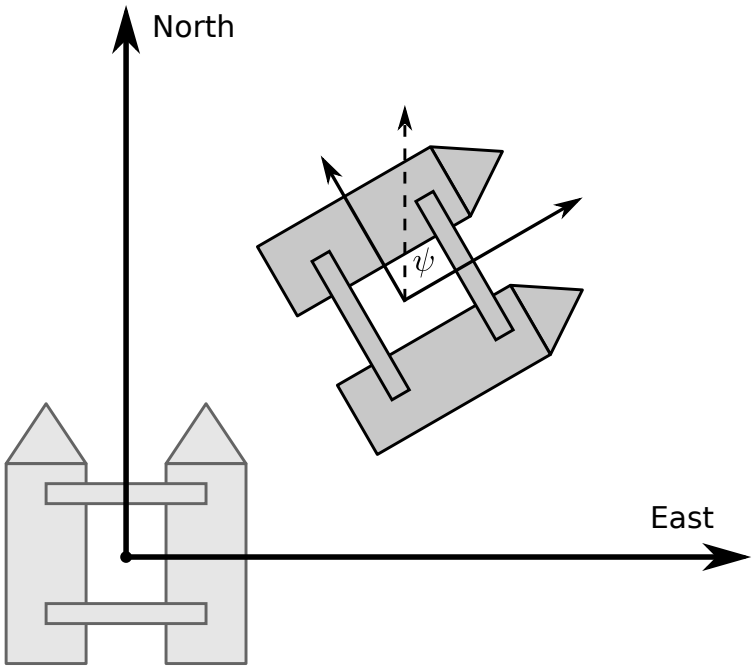

Figure 5.1: Piranha navigation model and frame.

5.1.1 Equation of Motion

We can find the EOM of the navigation model:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{D}(\boldsymbol{\nu}, \boldsymbol{w}) = \boldsymbol{\tau} \tag{5.1}$$

Because $z,\ \phi,\ \theta$ are removed, only $p_n,\ p_e,\ \psi$ remains in the equation. The position state vector $\boldsymbol{x}_n$ and the velocity vector are defined as:

$$\boldsymbol{x}_n = \begin{bmatrix} p_n \\ p_e \\ \psi \end{bmatrix} \quad \boldsymbol{v}_n = \begin{bmatrix} u \\ v \\ r \end{bmatrix} \tag{5.2}$$

5.1.2 Control Input

The input vector is the same as the simulation EOM:

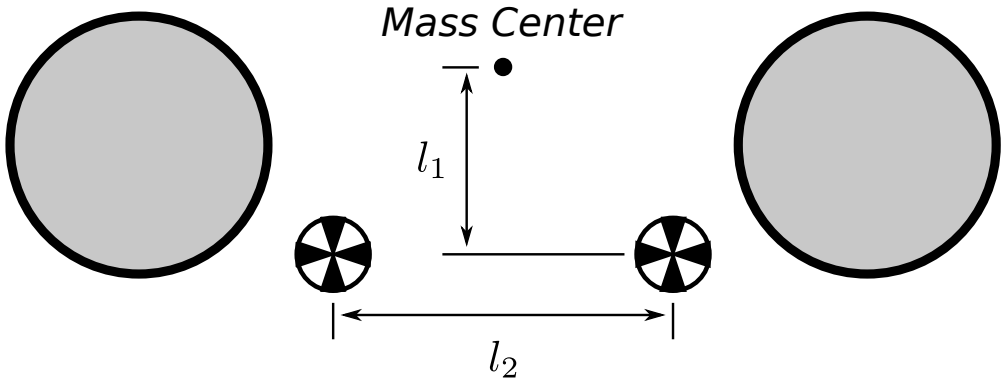

Figure 5.2: Thrusters' location with respect to the mass center.

$$\boldsymbol{\tau} = \begin{bmatrix} f_u \\ 0 \\ t_\tau \end{bmatrix} = \boldsymbol{T} \begin{bmatrix} u_f \\ u_r \end{bmatrix} = \boldsymbol{T}\boldsymbol{F}_\Delta(\boldsymbol{u}_{\text{PWM}}) \tag{5.3}$$

Here a new input pair $[f_f, t_r]$ is used to simplify the formulation, f_f is the forward force, t_r is the input torque, we can solve back from this new input vector to the original input vector simply. And with the new input representation, we have:

$$\boldsymbol{\tau} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} f_f \\ t_r \end{bmatrix} \tag{5.4}$$

The reason why here a force-torque vector is the used, rather than the original PWM command vector, is that it does not break the linearity of the EOM, and on the other hand, we can search the PWM value the controller need back from the mapping function $\boldsymbol{F}_{\text{map}}$.

$$\boldsymbol{u}_{\text{PWM}} = \boldsymbol{F}_{\text{map}}^{-1} \left(\begin{bmatrix} u_f \\ u_r \end{bmatrix} \right) \tag{5.5}$$

5.1.3 Mass matrix

The mass matrix can be written as:

$$\boldsymbol{M} = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & I \end{bmatrix} \tag{5.6}$$

Where m is the mass value of Piranha, and I is the z axis rotational inertia.

5.1.4 Damping Matrix

$\boldsymbol{D}(\boldsymbol{\nu})$ is the damping matrix and is much simpler than the one in simulation. We formulate it as:

$$\boldsymbol{D}(\boldsymbol{\nu}) = \begin{bmatrix} -\frac{1}{2}A_f\rho_d u^2 \\ -\frac{1}{2}A_s\rho_d v^2 \\ -\frac{1}{2}A_I\rho_r r^2 \end{bmatrix} \tag{5.7}$$

Where A_f, A_s, A_I represent the Piranha's frontal area for u v and r direction. ρ_d is the second order drag coefficient and ρ_r is the second order rotational drag coefficient.

In the end, the whole EOM can be found as:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \boldsymbol{\nu} \tag{5.8}$$

$$\dot{\boldsymbol{\nu}} = \frac{1}{M}\left(\begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{F}_{\Delta}(\boldsymbol{u}_{\mathrm{PWM}}) - \begin{bmatrix} -\frac{1}{2}A_f\rho_d u^2 \\ -\frac{1}{2}A_s\rho_d v^2 \\ -\frac{1}{2}A_I\rho_r r^2 \end{bmatrix} \right) \tag{5.9}$$

We can see that only the u and ψ are controllable. The side-way speed, v, is constantly deteriorating because of the resistance.

5.2 Extended Kalman Filter

Piranha is equipped with multiple sensors, including two accelerometers, two gyroscopes, three magnetometers, one GPS. In such a situation, we developed an Extended Kalman Filter (EKF) for Piranha to use all the measurements to estimate the boat's position, velocity, and orientation. The full python script is attached to the Appendix A for a better understanding of the structure of the EKF on Piranha. The mathematical formulation follows the latest EKF paper [2], and I followed the software design criteria written on the engineering practice [13, 14].

Two EKF instances run parallel on the two IMUs and output two results for the estimated states. The final result of the estimator depends on the ICM20948 EKF instance. However, another value called result credibility is calculated based on the difference between the two EKF outputs, and it decides if the EKF result is valid in the run time.

5.2.1 States and Observations

The EKF used on Piranha has 13 state variables in the state vector. These variables include:

- $[q_0, q_1, q_2, q_3]$ 4 quaternions which define the orientation.
- $[v_n, v_e, v_d]$ NED velocity vector.
- $[p_n, p_e, p_d]$ NED position vector.
- $[\text{mag}_N, \text{mag}_E, \text{mag}_D]$ NED earth fixed magnetic field components.

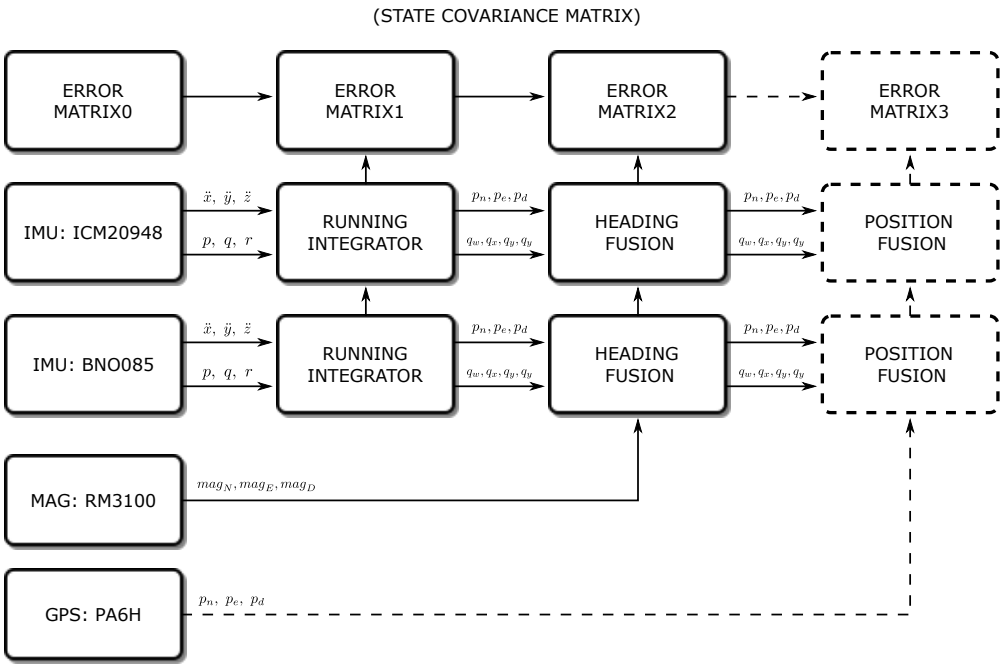

Figure 5.3: The overall EKF flowchart on Piranha.

5.2.2 State Predict

After getting a new set of raw data from the IMUs, the algorithm first predicts the update on the quaternions q_0, q_1, q_2, q_3, and the position p_n, p_e, p_d in the NED frame. First the program does the following steps:

1. Get raw readings from each IMU. These raw readings include the acceleration values $\Delta v_x, \Delta v_y, \Delta v_z$ from each accelerometer, and the angular speed values $\Delta a_x, \Delta a_y, \Delta a_z$ from each gyroscope.

2. Feed the raw readings to the running integrators, save the previously estimated state vector, and check the predicted state vector's output. During this time, the values in the error matrix (sometimes called state covariance matrix $P[t+1]$) also grow.

IMU Prediction Equation

The observation variables of the IMU includes:

- $[\Delta a_x, \Delta a_y, \Delta a_z]$ Delta angular positions in the XYZ frame (xyz accelerations).

- $[\Delta v_x, \Delta v_y, \Delta v_z]$ Delta velocity in the XYZ frame (xyz angular velocities).

We can write the prediction equations or the discretized EOM as:

$$\begin{bmatrix} v_n \\ v_e \\ v_d \end{bmatrix}_{[t+1]} = \begin{bmatrix} v_n \\ v_e \\ v_d \end{bmatrix}_{[t]} + \begin{bmatrix} g_n \\ g_e \\ g_d \end{bmatrix}_{[t]} \Delta t + Q(q_0, q_1, q_2, q_3) * \begin{bmatrix} \Delta v_x \\ \Delta v_y \\ \Delta v_z \end{bmatrix} \Delta t_{[t]} \tag{5.10}$$

$$\begin{bmatrix} p_n \\ p_e \\ p_d \end{bmatrix}_{[t+1]} = \begin{bmatrix} p_n \\ p_e \\ p_d \end{bmatrix}_{[t]} + \begin{bmatrix} v_n \\ v_e \\ v_d \end{bmatrix}_{[t]} \Delta t \tag{5.11}$$

$$\begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}_{[t+1]} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}_{[t]} \bigotimes \begin{bmatrix} \cos\frac{\theta}{2} \\ \frac{\boldsymbol{\omega}_x}{\|\boldsymbol{\omega}\|}\sin\frac{\theta}{2} \\ \frac{\boldsymbol{\omega}_y}{\|\boldsymbol{\omega}\|}\sin\frac{\theta}{2} \\ \frac{\boldsymbol{\omega}_z}{\|\boldsymbol{\omega}\|}\sin\frac{\theta}{2} \end{bmatrix}_{[t]} \approx \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}_{[t]} \bigotimes \begin{bmatrix} 1 \\ 0.5\Delta a_x \Delta t \\ 0.5\Delta a_y \Delta t \\ 0.5\Delta a_z \Delta t \end{bmatrix}_{[t]} \tag{5.12}$$

The $[\text{mag}_N, \text{mag}_E, \text{mag}_D]$ prediction equation is very simple. As we know, the earth's magnetic field barely changes locally, so the EKF predicts that these three values keep constant:

$$\begin{bmatrix} \tilde{\text{mag}}_N \\ \tilde{\text{mag}}_E \\ \tilde{\text{mag}}_D \end{bmatrix}_{[t+1]} = \begin{bmatrix} \text{mag}_N \\ \text{mag}_E \\ \text{mag}_D \end{bmatrix}_{[t]} \tag{5.13}$$

Where g_n, g_e, g_d are the gravity components in NED frame, $\mathcal{F}$ is the quaternion multiplication function.

After getting the discretized EOM, EKF calculates the Jacobian of the EOM $\boldsymbol{F}_j$ using the built-in python function `jacobian()`. Then it is used to update the covariance matrix:

$$\boldsymbol{P}[t+1] = \boldsymbol{F}_j \boldsymbol{P}[t+1] {\boldsymbol{F}^\top}_j + \boldsymbol{Q}_k \tag{5.14}$$

5.2.3 Attitude Update

Although the gyroscope provides information about the rotation angles, we need an extra step to align the starting position with the earth-fixed NED system to have the correct yaw angle. In other words, to make the North axis point to the geographical north pole. This alignment is essential for marine navigation. To do so, EKF needs a compass or magnetometer.

An extra highly sensitive magnetometer RM3100 is installed at the boat's head to generate precise yaw angle measurement.

When EKF sees a new magnetometer measurement, the measurement is in the body frame, so the observation function is:

$$\mathbf{mag}_{ob} = h(\boldsymbol{x}_{\text{EKF}}) = R_{\text{quat}}(q_0, q_1, q_2, q_3)^\top \begin{bmatrix} \text{mag}_\text{N} \\ \text{mag}_\text{E} \\ \text{mag}_\text{D} \end{bmatrix}_{[t+1]} \tag{5.15}$$

The predicted observation:

$$\tilde{\mathbf{mag}}_{ob} = h(\boldsymbol{x}_{\text{EKF}}) = R_{\text{quat}}(\tilde{q_0}, \tilde{q_1}, \tilde{q_2}, \tilde{q_3})^\top \begin{bmatrix} \tilde{\text{mag}}_N \\ \tilde{\text{mag}}_E \\ \tilde{\text{mag}}_D \end{bmatrix}_{[t+1]} \tag{5.16}$$

EKF updates the attitude through:

$$\Delta \mathbf{mag}_{ob} = \mathbf{mag}_{ob} - \tilde{\mathbf{mag}}_{ob} \tag{5.17}$$

$$\mathbf{H}_k = \left.\frac{\partial h}{\partial \boldsymbol{x}}\right|_{\tilde{x}_k} \tag{5.18}$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_k \mathbf{H}_k^\top + \mathbf{R}_k \tag{5.19}$$

$$\mathbf{K}_k = \mathbf{P}_k \mathbf{H}_k^\top \mathbf{S}^{-1} \tag{5.20}$$

$$\tilde{\boldsymbol{x}}_k = \tilde{\boldsymbol{x}}_k + \mathbf{K}_k \Delta \tilde{\mathbf{mag}}_{ob} \tag{5.21}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)\mathbf{P}_k \tag{5.22}$$

It will update both the stored magnetic field vector $\text{mag}_N, \text{mag}_E, \text{mag}_D$ and the quaternions $[q0, q1, q2, q3]$.

Body Magnetic Field

In the discussion above, the calculation only considers the earth's magnetic field. However, in reality, the circuit on Piranha will inevitably create a magnetic field in the body frame. Adding the body's magnetic vector to the state vector list reduces the noise of the attitude update. However, because we installed the magnetometer on Piranha far from the electrical parts, we ignored the effect of the body's magnetic field.

5.2.4 Position Update

EKF enters this step when there is a new position reading on the GPS sensor, a standard GPS message **$GPSGGA** of the NMEA format contains the following information:

- UTC Time.
- Latitude.

- North/South Indicator.
- Longitude.
- West/East Indicator.

The NED position measurement $p_{n\text{GPS}},\ p_{e\text{GPS}},\ p_{d\text{GPS}}$ is calculated based on the latitude and longitude values first. Then it will be the observation state:

$$\boldsymbol{y}_{ob} = \begin{bmatrix} p_n \\ p_e \\ p_d \end{bmatrix} \tag{5.23}$$

So the innovation step is:

$$\Delta \boldsymbol{y}_{ob} = \boldsymbol{y}_{ob} - \tilde{\boldsymbol{y}}_{ob} \tag{5.24}$$

$$\mathbf{H}_k = \left.\frac{\partial h}{\partial \boldsymbol{x}}\right|_{\tilde{x}_k} \tag{5.25}$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_k \mathbf{H}_k^\top + \mathbf{R}_k \tag{5.26}$$

$$\mathbf{K}_k = \mathbf{P}_k \mathbf{H}_k^\top \mathbf{S}^{-1} \tag{5.27}$$

$$\tilde{\boldsymbol{x}}_k = \tilde{\boldsymbol{x}}_k + \mathbf{K}_k \Delta \boldsymbol{y}_{ob} \tag{5.28}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)\mathbf{P}_k \tag{5.29}$$

Because GPS measurements are not always available, this step will be skipped if the GPS module loses fix on the satellite and fails to obtain a new measure. In such a situation, the error matrix P_k starts growing in the repeated predict steps, meaning the error accumulates in the estimated states.

5.2.5 EKF Results

Figure 5.4 and 5.5 show two EKF estimated trajectories and the raw GPS test data. We did these tests using the simulation program discussed in the previous section. We can see that, in the simulation, the EKF on Piranha can estimate the trajectory correctly even with huge GPS noise.

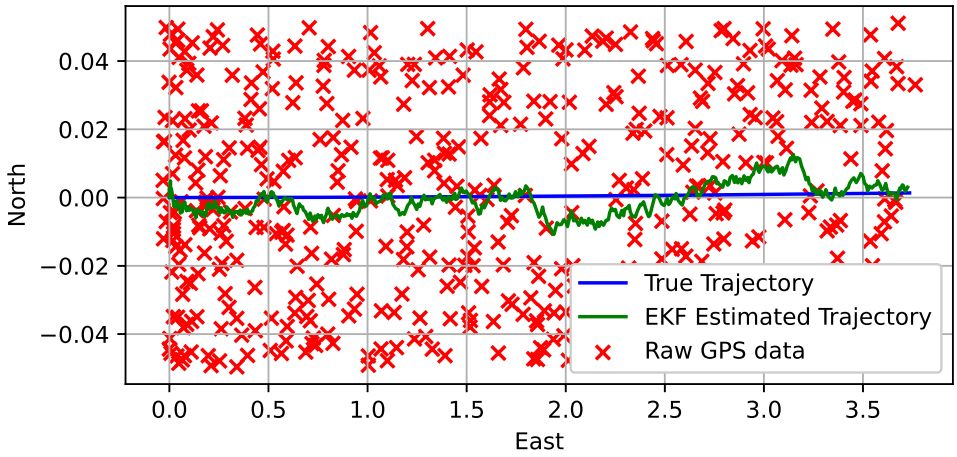

Figure 5.4: EKF estimated trajectory versus raw GPS data in the linear driving simulation.

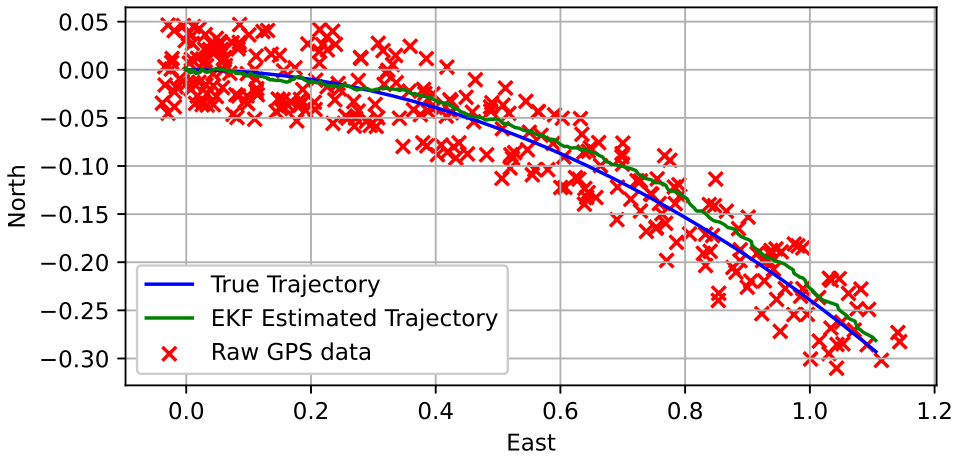

Figure 5.5: EKF estimated trajectory versus raw GPS data in the steering simulation.

5.3 PD Controller for Yaw Control

Because there are errors in many ways, like we did not properly align the thrusters during the assembly, or one thruster is weaker than the other, the boat cannot drive in a straight line without a closed-loop controller. So a controller for yaw control driving is needed. On Piranha, A simple PD controller is implemented for this purpose.

The key to linear driving is to keep the headings stable. We can see the structure of the PD controller in Figure 5.6.

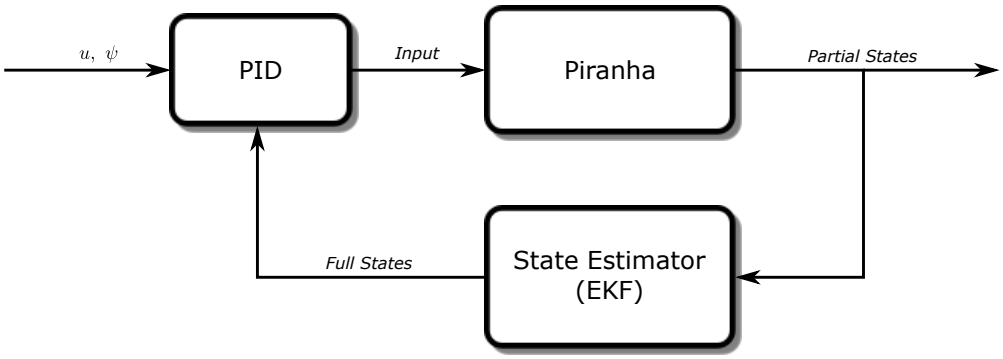

Figure 5.6: Piranha PD heading controller flowchart.

The PD equation is:

$$f_f = k_{p1}(u - \tilde{u}) \tag{5.30}$$

$$t_r = k_{p2}(0 - \tilde{\psi}) + k_{p3}(0 - \tilde{r}) + k_d(0 - \frac{\mathrm{d}\tilde{r}}{\mathrm{d}t}) \tag{5.31}$$

After getting the f_f and t_r, the controller solves the raw input using the reversed input mapping function:

5.3.1 Test Results

A simple test can show how the PD controller works. In the test, we reduced the left thruster's maximum power output to 80%. Meanwhile, the right thruster is still at 100% of the original. In real life, it is common that some ropes or plastic wastes get caught on the thruster protector and drag the left thruster down, or it is simply because the output power is different between the two thrusters due to design defections. However, we can see in Figure 5.7, 5.8, 5.9, 5.10, 5.11 that without a PD controller, the heading ψ keeps increasing, making the boat drive to the left.

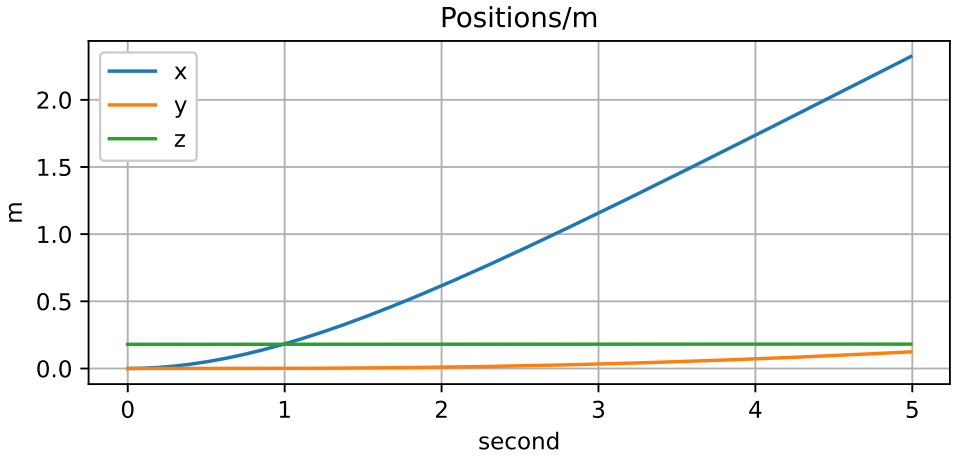

Figure 5.7: Simulation: Without the PD controller - Positions x, y, z.

With the PD controller, the input of the two thrusters are automatically calculated based on the observation to decrease the error in the heading ψ. The results improved by a lot, the ψ stays within a small deviation range, and the drift on the y direction is eliminated. The test results can be seen in Figure 5.12, 5.13, 5.14, 5.15, 5.16.

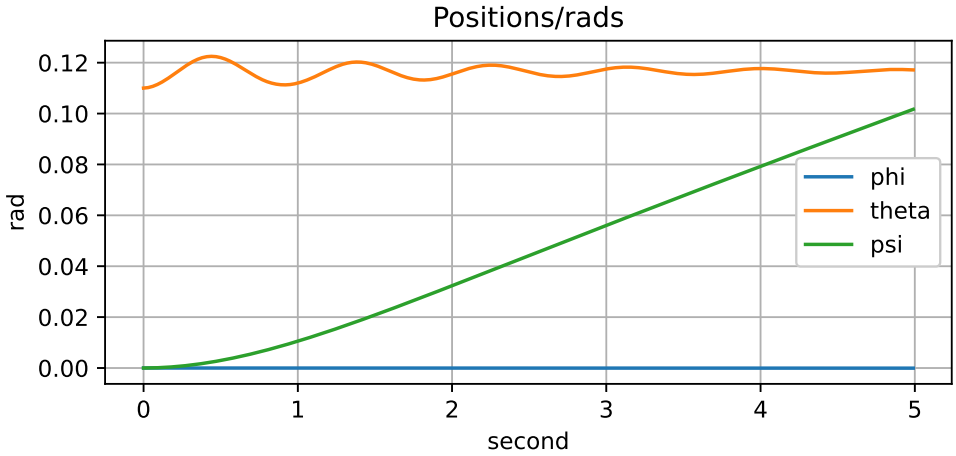

Figure 5.8: Simulation: Without the PD controller - Positions ϕ, θ, ψ.

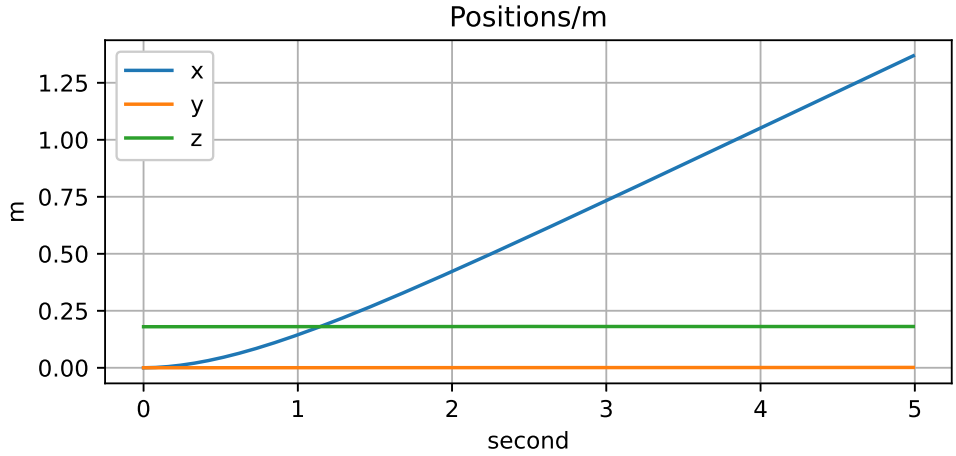

Figure 5.12: Simulation: With the PD controller - Positions x, y, z.

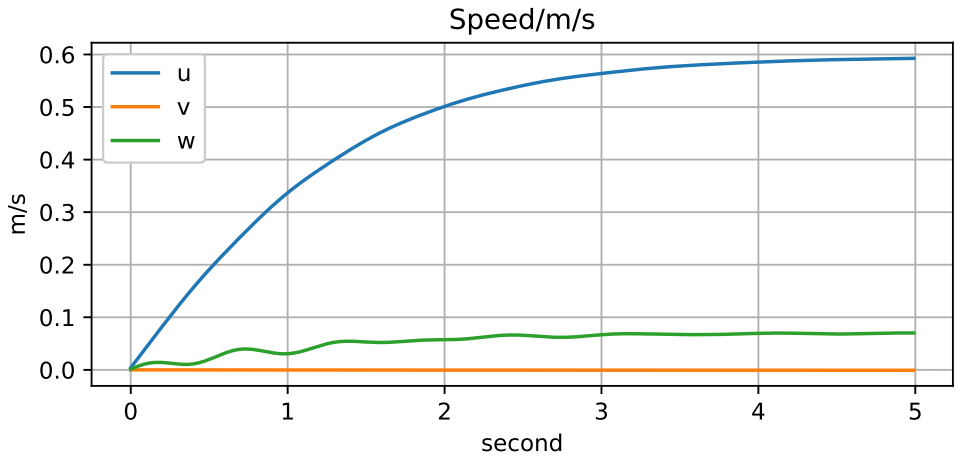

Figure 5.9: Simulation: Without the PD controller - Linear Speed u, v, w.

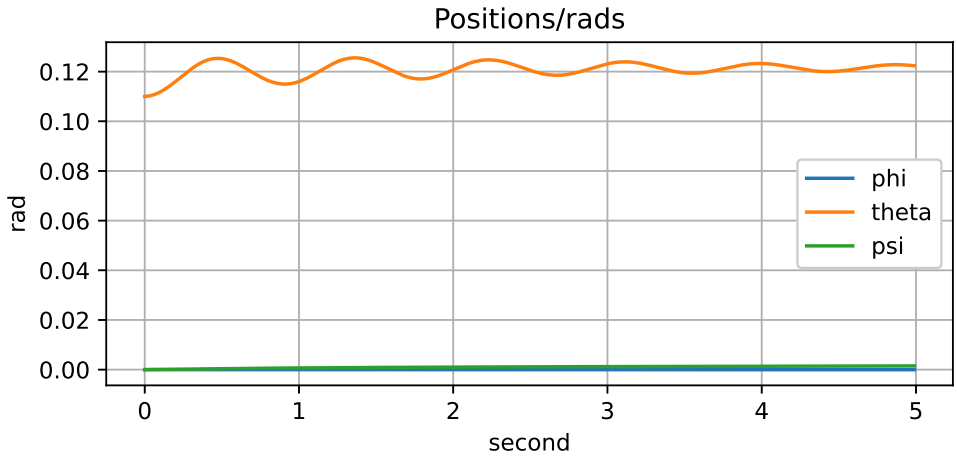

Figure 5.13: Simulation: With the PD controller - Positions ϕ, θ, ψ.

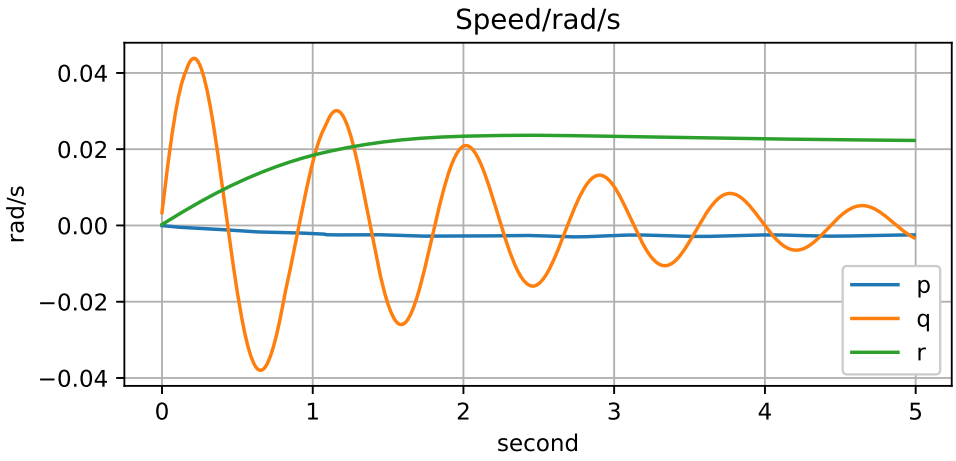

Figure 5.10: Simulation: Without the PD controller - Angular Speed p, q, r.

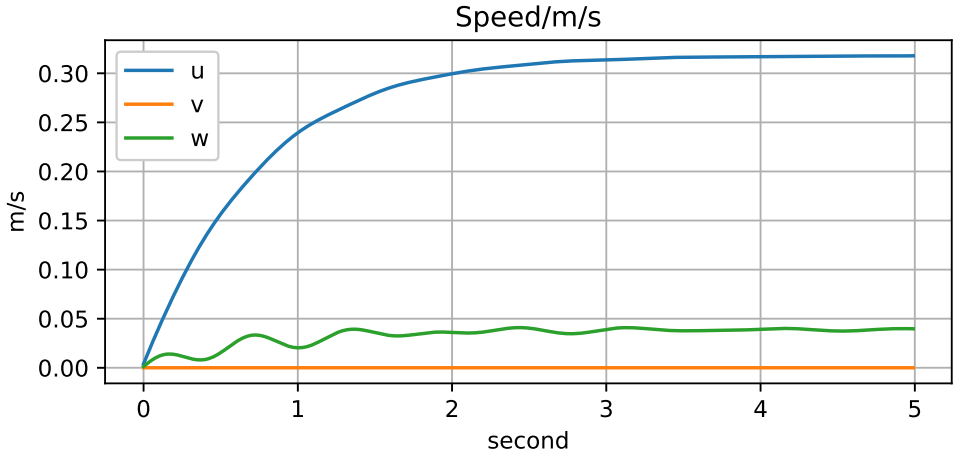

Figure 5.14: Simulation: With the PD controller - Linear Speed u, v, w.

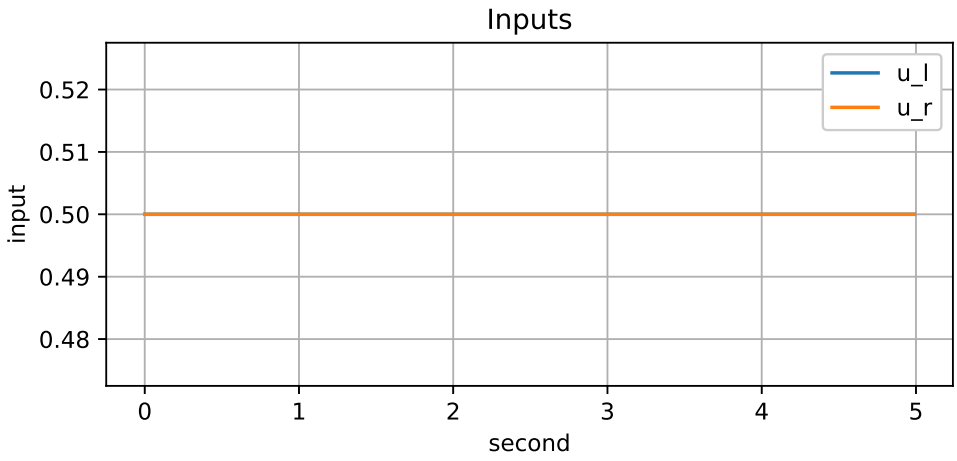

Figure 5.11: Simulation: Without the PD controller - Inputs u_l, u_r.

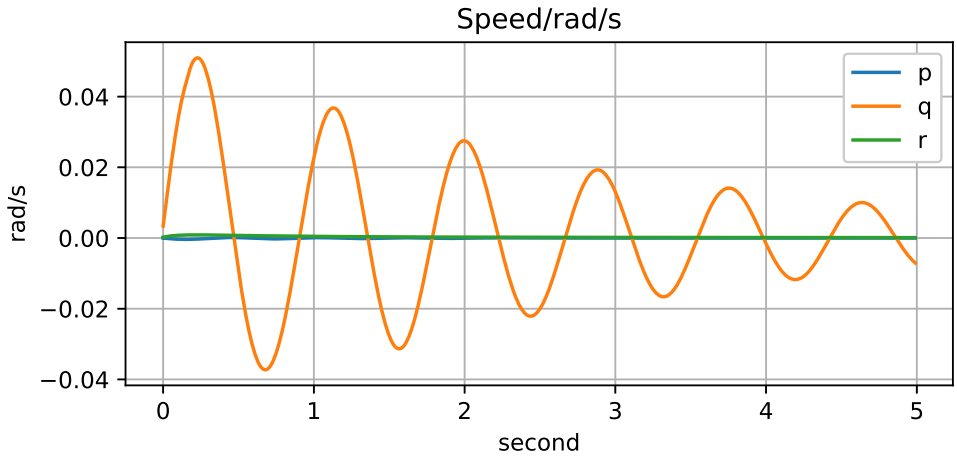

Figure 5.15: Simulation: With the PD controller - Angular Speed p, q, r.

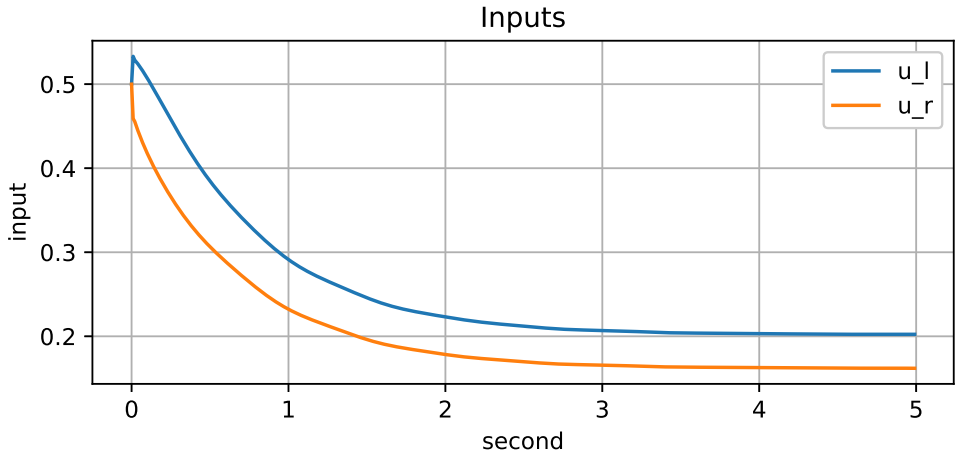

Figure 5.16: Simulation: With the PD controller - Inputs u_l, u_r.

5.4 Trajectory Tracking

In the previous section, the PD controller can control the forward speed u and the heading position ψ. However, the PD controller itself is not enough for Piranha to track a trajectory. This section explains the experimental trajectory tracking controller on Piranha. It was designed based on the feedback linearization theory. However, it did have the ability to correct the error one a specific direction.

In the future, I am going to test reinforcement learning and model predictive control. Previous research showed good results by using these two methods[19], [5], [9].

5.4.1 Feedback Linearization Controller

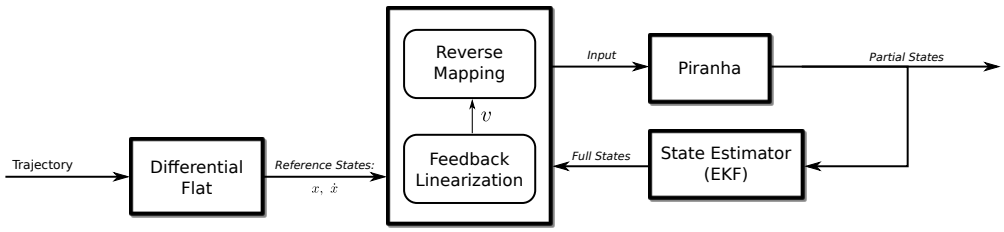

Figure 5.17: Piranha's trajectory tracking feedback linearization controller flowchart.

Readers can check the feedback linearization algorithm workflow in Figure 5.17. The previous section brought the EOM of the navigation model, which can be simplified as:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \boldsymbol{\nu} \tag{5.32}$$

$$\dot{\boldsymbol{\nu}} = \frac{1}{M} \left(\begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{u}_f - \begin{bmatrix} -\frac{1}{2} A_f \rho_d u^2 \\ -\frac{1}{2} A_s \rho_d v^2 \\ -\frac{1}{2} A_I \rho_d r^2 \end{bmatrix} \right) \tag{5.33}$$

To construct the feedback linearization vector, we can construct the state vector as:

$$\boldsymbol{x}_l = \begin{bmatrix} p_n \\ p_e \\ \psi \end{bmatrix} \tag{5.34}$$

The observation vector is the same as the state vector with the EKF estimator brought in in the previous section.

$$\boldsymbol{y}_l = \boldsymbol{x}_l \tag{5.35}$$

Since it is a second-order system, construct a feedback vector $\boldsymbol{z}$, so that:

$$\boldsymbol{z} = T(\boldsymbol{x}_l) = \begin{bmatrix} \boldsymbol{x}_l \\ \\ \dot{\boldsymbol{x}}_l \end{bmatrix} \tag{5.36}$$

We can find the $\dot{\boldsymbol{x}}_l$ from the early EOM. Because we can directly observe the velocity vector $\boldsymbol{\nu}$ in the body frame, so its vector form is reserved. The $\ddot{\boldsymbol{x}}_l$ can be found as:

$$\ddot{\boldsymbol{x}}_l = \begin{bmatrix} -\sin\dot{\psi} & -\cos\dot{\psi} & 0 \\ \cos\dot{\psi} & -\sin\dot{\psi} & 0 \\ 0 & 0 & 0 \end{bmatrix} \boldsymbol{\nu} + \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \dot{\boldsymbol{\nu}} \tag{5.37}$$

In the $\dot{\boldsymbol{\nu}}$ the input vector can be found. Define the new input of the system to be $\boldsymbol{v}$, So that the $\boldsymbol{v}$ is a linear function of $\boldsymbol{u}_f$:

$$\ddot{\boldsymbol{x}}_l = f_l(\boldsymbol{u}_f) = \boldsymbol{A}_l \cdot \boldsymbol{u}_f + \boldsymbol{b}_l \tag{5.38}$$

Which turns the given system into a linearized system with a cascade of integrators:

$$\begin{aligned} \dot{\boldsymbol{z}}_1 &= z_2 \\ \dot{\boldsymbol{z}}_2 &= \ddot{\boldsymbol{x}}_l \end{aligned} \tag{5.39}$$

By the control law, in such a linearized system, the control vector v can be found as:

$$\boldsymbol{v} = -K\boldsymbol{z} \tag{5.40}$$

After finding the control vector $\boldsymbol{v}$, the original control vector $\boldsymbol{u}_f$ can be found using the reverse mapping from $u_f \to v$.

$$\boldsymbol{u}_f = A^{+}(\boldsymbol{v} - \boldsymbol{b}) \tag{5.41}$$

Because the matrix A is not invertible, A^+ denotes the pseudo-inverse of the A matrix. However, the primary error of the feedback linearization controller arises in this step. To explain it, we first find that:

$$\begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}^+ = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{5.42}$$

The equation means the error in the second variable of the state $[u,\ v,\ w]$, i.e., the velocity at the v direction is ignored while multiplying this matrix. We can explain this error by the fact that the system cannot control the v variable. As a result, the error in the side-way direction accumulated during the trajectory, which can be seen in the next section.

Circle Trajectory Result

The first test is the circle trajectory test, the results are showed in the following Figures 5.18, 5.19, 5.20,5.21,5.22, 5.23.

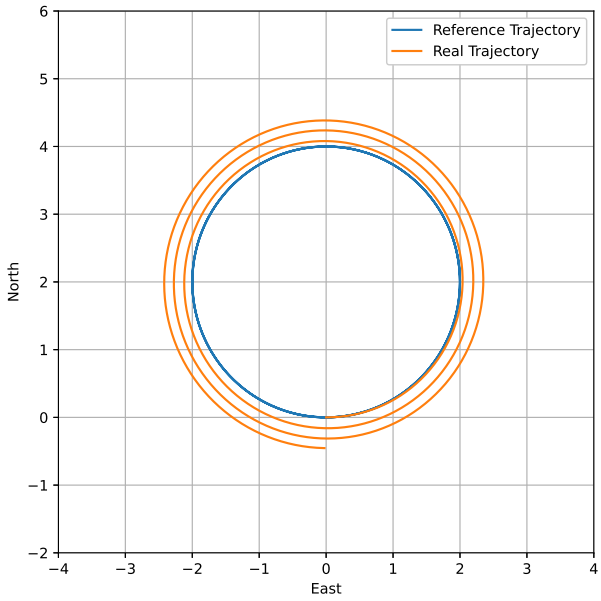

Figure 5.18: Feedback linearization trajectory tracking controller test result, the reference is a circle trajectory.

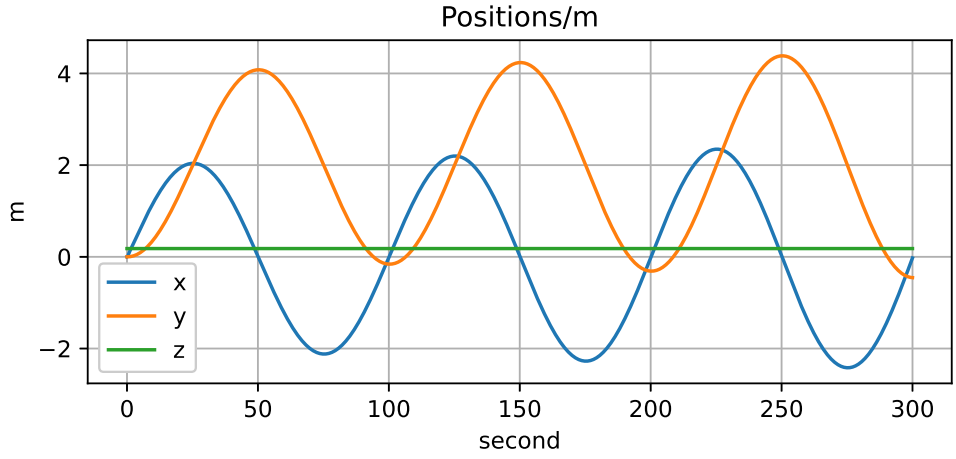

Figure 5.19: Feedback linearization trajectory tracking controller test result - Position $x,\ y,\ z$.

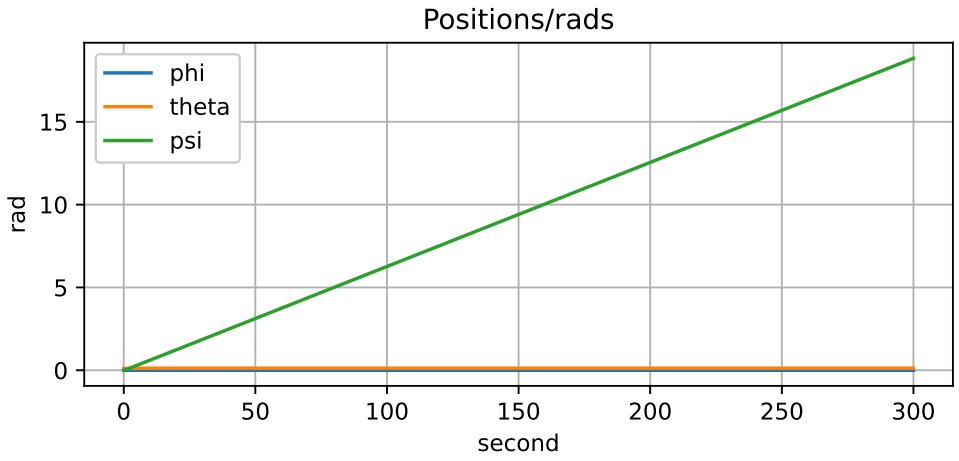

Figure 5.20: Feedback linearization trajectory tracking controller test result - Attitude $\phi,\ \theta,\ \psi$.

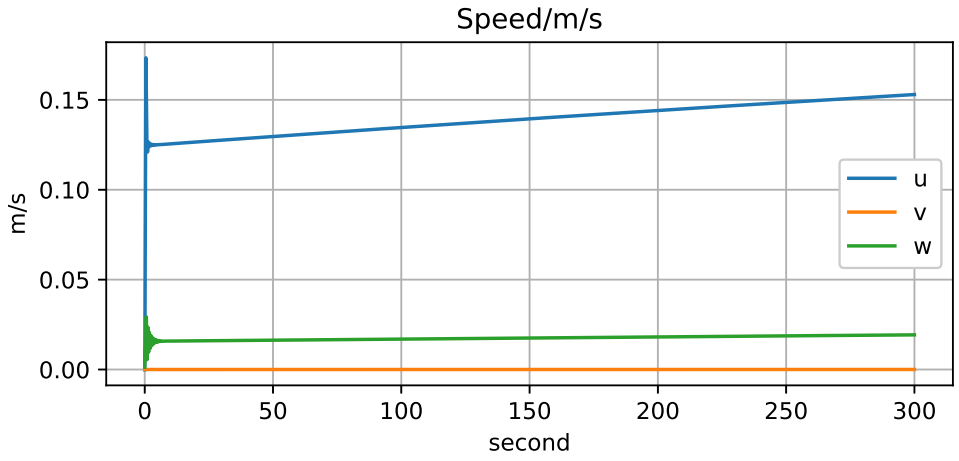

Figure 5.21: Feedback linearization trajectory tracking controller test result - Linear Speed $u,\ v,\ w$.

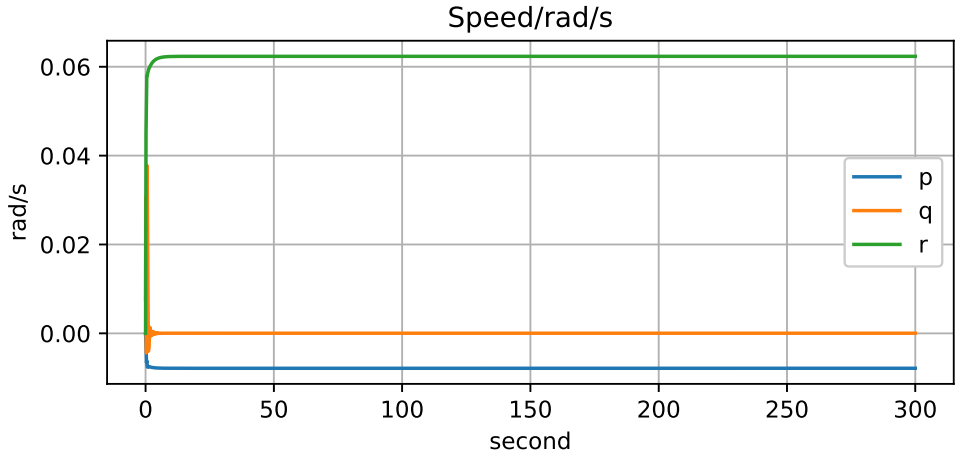

Figure 5.22: Feedback linearization trajectory tracking controller test result - Angular Speed $p,\ q,\ r$.

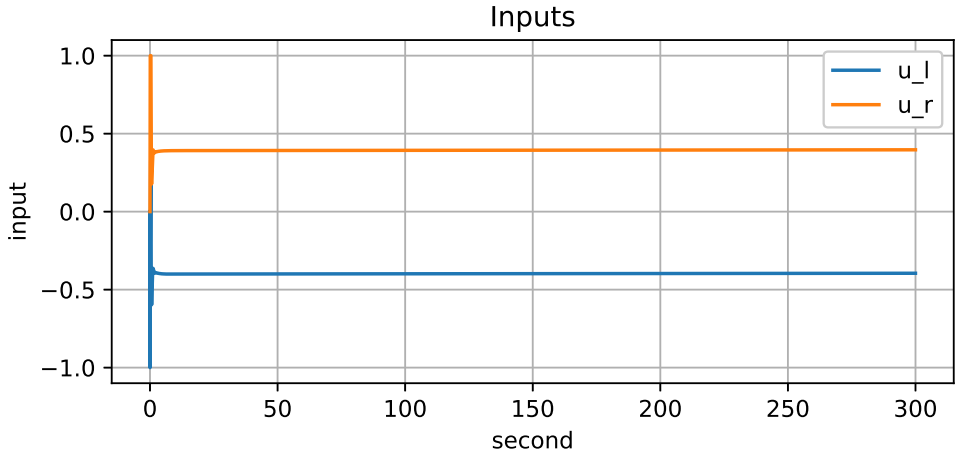

Figure 5.23: Feedback linearization trajectory tracking controller test result - Input $u_l,\ u_r$.

Square Trajectory Result

The second test has a square-shaped trajectory. Compared with the circle trajectory, there are discontinuities at each vertex of the square at the reference velocity and heading. In this test, we can see that the feedback linearization controller can correct the yaw angle. However, it cannot reduce the error in the v direction since the input $u_l,\ u_r$ does not control v directly.

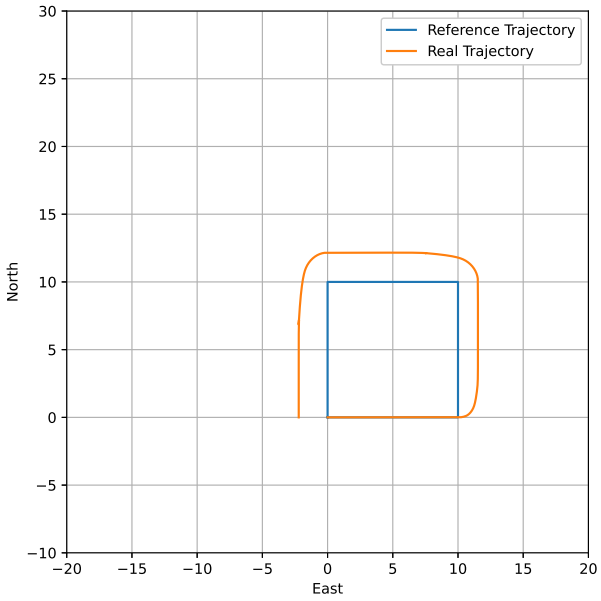

Figure 5.24: Feedback linearization trajectory tracking controller test result, the reference is a square trajectory.

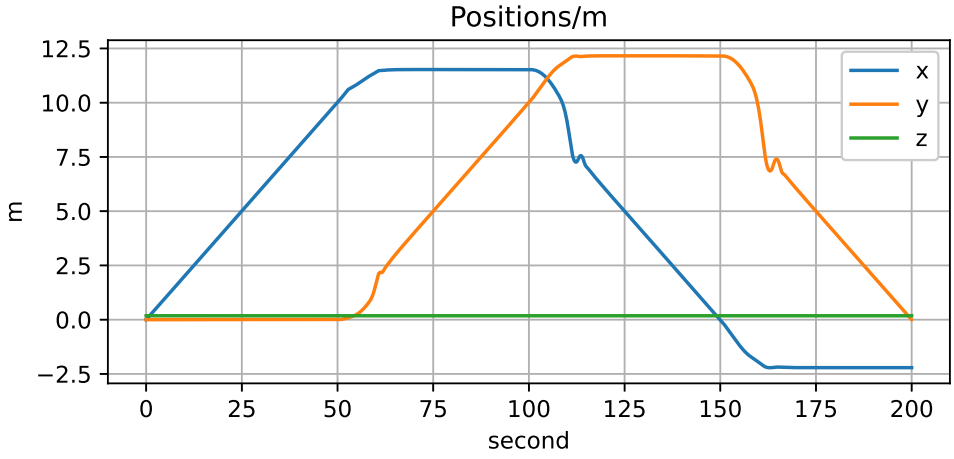

Figure 5.25: Feedback linearization trajectory tracking controller test result - Position x, y, z.

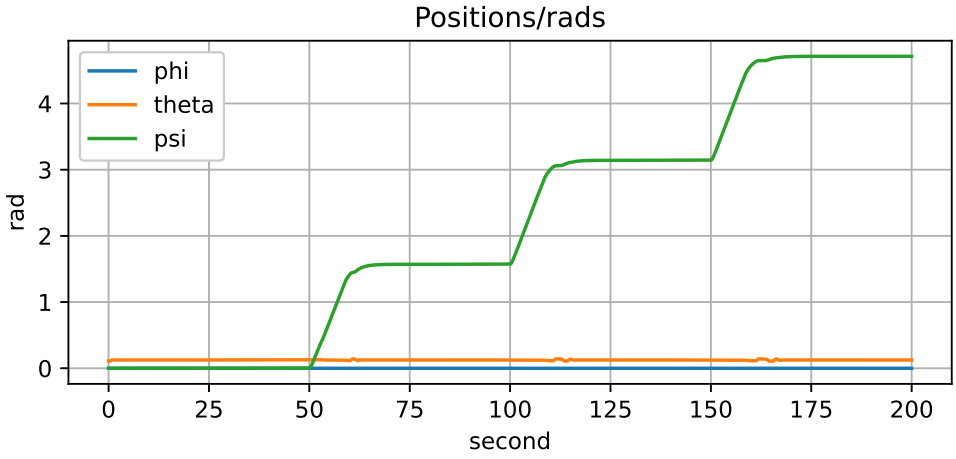

Figure 5.26: Feedback linearization trajectory tracking controller test result - Attitude ϕ, θ, ψ.

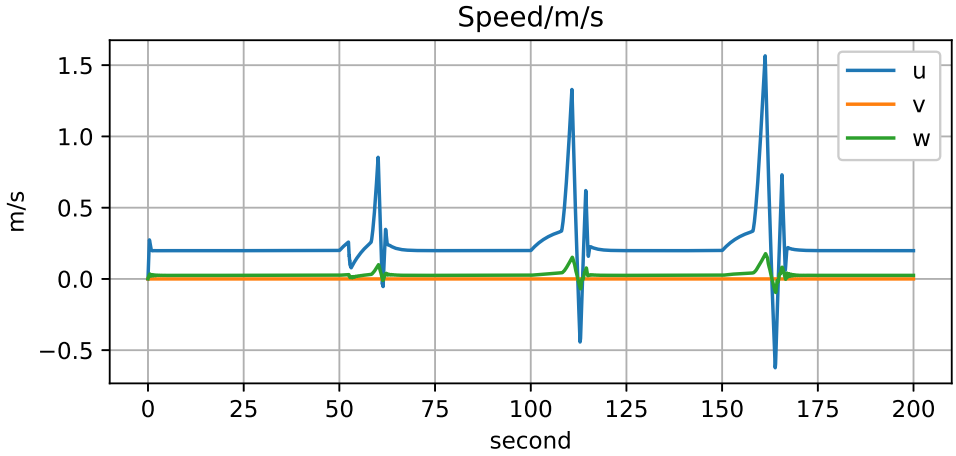

Figure 5.27: Feedback linearization trajectory tracking controller test result - Linear Speed u, v, w.

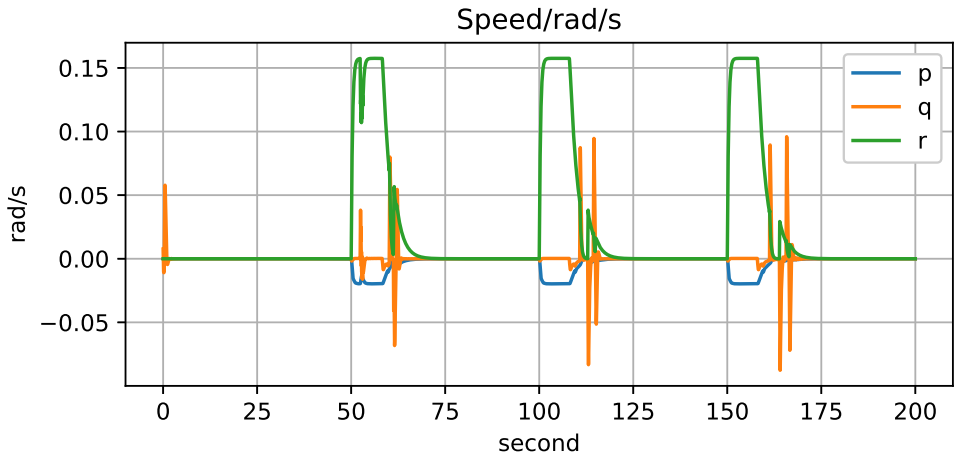

Figure 5.28: Feedback linearization trajectory tracking controller test result - Angular Speed $p,\ q,\ r$.

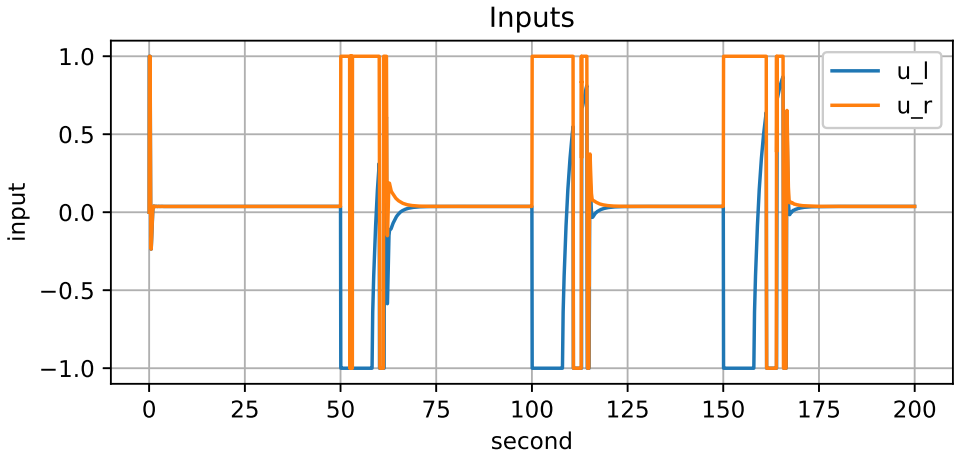

Figure 5.29: Feedback linearization trajectory tracking controller test result - Input $u_l,\ u_r$.

We can see that the feedback linearization controller can navigate Piranha to drive towards the expected trajectory. However, because it only considers the error at present, it cannot eliminate the error at the v direction, which requires the controller to have the ability to plan.

Chapter 6

Conclusion

This report describes the water surface robot, Piranha. The first chapter covers the initial concept of Piranha and compares multiple available trash collection options before Piranha's configuration is finally determined. The second chapter tells the reader how we built the Piranha from simple mechanical parts. The third chapter illustrates the electrical system that powers the Piranha and the specifications of all components and their verification. The fourth chapter proposes a new way to formulate a dynamic model for a water surface vehicle and provides the numerical methods needed on the computer. The fifth chapter first introduces a 13-state EKF based on the modern control theory and then a PD controller for heading and speed control. At the end of the fifth chapter, this report describes a feedback linearization controller to achieve the trajectory tracking function on Piranha.

The simulation brought in chapter four is based on several existing designs. Because it reads the model as a triangle mesh, it does not care about the configurations of the watercraft. Additionally, the simulation results have a very high precision because it uses a more precise frontal area estimate algorithm based on the sliced triangle faces.

We keep a balance between complexity and speed for the EKF discussed in chapter five. It

shows good stability even under considerable GPS noise. There is a possibility of increasing the state vector length and acquiring better precision. However, the EKF will also lose its speed simultaneously.

This report also serves as an entry-level engineering practice guidance for anyone who wants to build an RC vehicle, either a car, drone, or boat, in the real world. It includes concept development, mechanical design, electrical design, simulation, and the control system design, all essential sections for a successful robot design.

6.1 Future work

Although it is still at prototyping stage, Piranha will be a commercial product and eventually be on the market. So the most critical work for the mechanical and electrical design in the next step is to simplify the manufacturing process and speed up the production meanwhile reducing the fabrication cost for each unit. This work includes customizing plastic molds and drawing PCBs.

On the safety aspect, electromagnetic interference (EMI) and several fail-safe routines need to be designed in the next step to ensure the electrical system's stability and the operators' safety. Also, as a marine robot, it needs to meet the same industry standards for a common watercraft, such as the signal lights and other proper indicators.

There is still a lot of space for improvement in the control section, like using reinforcement learning and model predictive controllers for better tracking results. They are not discussed in this report because they are still under development at this moment. In the next step, more control approaches will be developed, and their performance will be investigated.

Appendix A

BlueRobotics Thruster T200 Data

PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)
1100	-2.9	1236	-1.56	1368	-0.52	1504	0	1636	0.68	1772	2.09
1104	-2.92	1240	-1.53	1372	-0.49	1508	0	1640	0.71	1776	2.13
1108	-2.89	1244	-1.49	1376	-0.47	1512	0	1644	0.76	1780	2.18
1112	-2.83	1248	-1.47	1380	-0.44	1516	0	1648	0.79	1784	2.24
1116	-2.79	1252	-1.44	1384	-0.42	1520	0	1652	0.83	1788	2.27
1120	-2.76	1256	-1.4	1388	-0.39	1524	0	1656	0.86	1792	2.33
1124	-2.72	1260	-1.37	1392	-0.37	1528	0	1660	0.89	1796	2.4
1128	-2.67	1264	-1.33	1396	-0.34	1532	0	1664	0.93	1800	2.46
1132	-2.6	1268	-1.29	1400	-0.32	1536	0	1668	0.97	1804	2.51
1136	-2.59	1272	-1.28	1404	-0.29	1540	0.04	1672	1	1808	2.56
1140	-2.56	1276	-1.22	1408	-0.27	1544	0.05	1676	1.04	1812	2.62
1144	-2.49	1280	-1.19	1412	-0.24	1548	0.07	1680	1.08	1816	2.65
1148	-2.44	1284	-1.15	1416	-0.23	1552	0.1	1684	1.14	1820	2.71
1152	-2.43	1288	-1.12	1420	-0.2	1556	0.11	1688	1.16	1824	2.78
1156	-2.39	1292	-1.08	1424	-0.18	1560	0.13	1692	1.2	1828	2.84
1160	-2.34	1296	-1.04	1428	-0.16	1564	0.15	1696	1.23	1832	2.87
1164	-2.3	1300	-1.02	1432	-0.15	1568	0.18	1700	1.28	1836	2.93
1168	-2.25	1304	-0.99	1436	-0.12	1572	0.2	1704	1.31	1840	3.01
1172	-2.23	1308	-0.96	1440	-0.11	1576	0.22	1708	1.35	1844	3.04
1176	-2.18	1312	-0.93	1444	-0.09	1580	0.25	1712	1.4	1848	3.08
1180	-2.14	1316	-0.9	1448	-0.07	1584	0.28	1716	1.43	1852	3.16
1184	-2.1	1320	-0.87	1452	-0.06	1588	0.31	1720	1.48	1856	3.23
1188	-2.07	1324	-0.83	1456	-0.05	1592	0.33	1724	1.53	1860	3.26
1192	-2.01	1328	-0.79	1460	-0.04	1596	0.37	1728	1.56	1864	3.32
1196	-1.98	1332	-0.77	1464	0	1600	0.39	1732	1.63	1868	3.38
1200	-1.95	1336	-0.74	1468	0	1604	0.43	1736	1.67	1872	3.4
1204	-1.88	1340	-0.72	1472	0	1608	0.46	1740	1.71	1876	3.45
1208	-1.85	1344	-0.69	1476	0	1612	0.49	1744	1.77	1880	3.5
1212	-1.81	1348	-0.66	1480	0	1616	0.52	1748	1.82	1884	3.57
1216	-1.78	1352	-0.64	1484	0	1620	0.55	1752	1.85	1888	3.64
1220	-1.73	1356	-0.6	1488	0	1624	0.59	1756	1.91	1892	3.71
1224	-1.66	1360	-0.57	1492	0	1628	0.63	1760	1.92	1896	3.69
1228	-1.65	1364	-0.54	1496	0	1632	0.65	1764	1.96	1900	3.71
1232	-1.61			1500	0			1768	2.03		

Figure A.1: T200 Public Performance Data 10-20V, 12 V

PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)
1100	-3.52	1236	-1.91	1368	-0.65	1504	0	1636	0.85	1772	2.56
1104	-3.5	1240	-1.89	1372	-0.62	1508	0	1640	0.89	1776	2.64
1108	-3.49	1244	-1.82	1376	-0.59	1512	0	1644	0.92	1780	2.66
1112	-3.45	1248	-1.76	1380	-0.55	1516	0	1648	0.97	1784	2.76
1116	-3.4	1252	-1.72	1384	-0.52	1520	0	1652	1	1788	2.78
1120	-3.36	1256	-1.68	1388	-0.49	1524	0	1656	1.05	1792	2.88
1124	-3.29	1260	-1.63	1392	-0.46	1528	0	1660	1.09	1796	2.93
1128	-3.25	1264	-1.58	1396	-0.43	1532	0	1664	1.14	1800	2.99
1132	-3.19	1268	-1.56	1400	-0.4	1536	0.05	1668	1.2	1804	3.05
1136	-3.14	1272	-1.52	1404	-0.37	1540	0.06	1672	1.23	1808	3.13
1140	-3.1	1276	-1.48	1408	-0.35	1544	0.08	1676	1.28	1812	3.19
1144	-3.06	1280	-1.44	1412	-0.32	1548	0.1	1680	1.32	1816	3.23
1148	-3	1284	-1.4	1416	-0.29	1552	0.12	1684	1.37	1820	3.32
1152	-2.94	1288	-1.37	1420	-0.26	1556	0.15	1688	1.41	1824	3.36
1156	-2.88	1292	-1.32	1424	-0.24	1560	0.18	1692	1.46	1828	3.42
1160	-2.85	1296	-1.28	1428	-0.21	1564	0.2	1696	1.51	1832	3.49
1164	-2.78	1300	-1.24	1432	-0.19	1568	0.23	1700	1.55	1836	3.57
1168	-2.76	1304	-1.19	1436	-0.16	1572	0.26	1704	1.61	1840	3.62
1172	-2.69	1308	-1.17	1440	-0.15	1576	0.29	1708	1.65	1844	3.69
1176	-2.64	1312	-1.12	1444	-0.12	1580	0.33	1712	1.71	1848	3.77
1180	-2.59	1316	-1.09	1448	-0.1	1584	0.36	1716	1.76	1852	3.84
1184	-2.53	1320	-1.05	1452	-0.08	1588	0.39	1720	1.81	1856	3.92
1188	-2.49	1324	-1.02	1456	-0.07	1592	0.43	1724	1.85	1860	3.98
1192	-2.45	1328	-0.98	1460	-0.05	1596	0.46	1728	1.91	1864	4.03
1196	-2.41	1332	-0.95	1464	-0.03	1600	0.5	1732	1.96	1868	4.11
1200	-2.35	1336	-0.92	1468	0	1604	0.53	1736	2	1872	4.15
1204	-2.34	1340	-0.88	1472	0	1608	0.58	1740	2.09	1876	4.21
1208	-2.26	1344	-0.85	1476	0	1612	0.62	1744	2.12	1880	4.3
1212	-2.2	1348	-0.81	1480	0	1616	0.64	1748	2.16	1884	4.38
1216	-2.18	1352	-0.77	1484	0	1620	0.69	1752	2.25	1888	4.42
1220	-2.12	1356	-0.74	1488	0	1624	0.73	1756	2.27	1892	4.51
1224	-2.05	1360	-0.7	1492	0	1628	0.77	1760	2.34	1896	4.53
1228	-2.03	1364	-0.68	1496	0	1632	0.83	1764	2.43	1900	4.52
1232	-1.99			1500	0			1768	2.5		

Figure A.2: T200 Public Performance Data 10-20V, 14 V

PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)
1100	-4.07	1236	-2.24	1368	-0.74	1504	0	1636	0.99	1772	2.98
1104	-4.05	1240	-2.2	1372	-0.72	1508	0	1640	1.03	1776	3.05
1108	-4.02	1244	-2.12	1376	-0.68	1512	0	1644	1.1	1780	3.11
1112	-3.96	1248	-2.08	1380	-0.65	1516	0	1648	1.14	1784	3.17
1116	-3.9	1252	-2.02	1384	-0.62	1520	0	1652	1.18	1788	3.22
1120	-3.87	1256	-1.98	1388	-0.58	1524	0	1656	1.24	1792	3.3
1124	-3.82	1260	-1.94	1392	-0.54	1528	0	1660	1.28	1796	3.37
1128	-3.8	1264	-1.86	1396	-0.51	1532	0.04	1664	1.33	1800	3.42
1132	-3.75	1268	-1.81	1400	-0.48	1536	0.05	1668	1.39	1804	3.51
1136	-3.71	1272	-1.76	1404	-0.44	1540	0.08	1672	1.44	1808	3.61
1140	-3.66	1276	-1.7	1408	-0.42	1544	0.1	1676	1.48	1812	3.68
1144	-3.59	1280	-1.67	1412	-0.39	1548	0.13	1680	1.54	1816	3.74
1148	-3.52	1284	-1.61	1416	-0.35	1552	0.15	1684	1.59	1820	3.82
1152	-3.45	1288	-1.56	1420	-0.32	1556	0.18	1688	1.65	1824	3.89
1156	-3.4	1292	-1.51	1424	-0.29	1560	0.22	1692	1.69	1828	3.96
1160	-3.31	1296	-1.49	1428	-0.26	1564	0.25	1696	1.76	1832	4.06
1164	-3.25	1300	-1.44	1432	-0.24	1568	0.29	1700	1.82	1836	4.15
1168	-3.2	1304	-1.4	1436	-0.21	1572	0.32	1704	1.88	1840	4.25
1172	-3.11	1308	-1.35	1440	-0.18	1576	0.36	1708	1.93	1844	4.3
1176	-3.04	1312	-1.3	1444	-0.15	1580	0.4	1712	1.99	1848	4.38
1180	-2.99	1316	-1.26	1448	-0.13	1584	0.44	1716	2.05	1852	4.51
1184	-2.94	1320	-1.2	1452	-0.1	1588	0.47	1720	2.12	1856	4.53
1188	-2.86	1324	-1.16	1456	-0.09	1592	0.51	1724	2.18	1860	4.65
1192	-2.82	1328	-1.12	1460	-0.07	1596	0.56	1728	2.22	1864	4.71
1196	-2.77	1332	-1.1	1464	-0.05	1600	0.6	1732	2.28	1868	4.79
1200	-2.71	1336	-1.05	1468	-0.04	1604	0.64	1736	2.38	1872	4.84
1204	-2.66	1340	-1.02	1472	0	1608	0.68	1740	2.43	1876	4.93
1208	-2.58	1344	-0.98	1476	0	1612	0.72	1744	2.52	1880	5.01
1212	-2.55	1348	-0.94	1480	0	1616	0.78	1748	2.58	1884	5.08
1216	-2.51	1352	-0.9	1484	0	1620	0.82	1752	2.65	1888	5.14
1220	-2.45	1356	-0.87	1488	0	1624	0.87	1756	2.73	1892	5.18
1224	-2.38	1360	-0.82	1492	0	1628	0.91	1760	2.76	1896	5.22
1228	-2.35	1364	-0.78	1496	0	1632	0.95	1764	2.84	1900	5.25
1232	-2.28			1500	0			1768	2.89		

Figure A.3: T200 Public Performance Data 10-20V, 16 V

PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)	PWM (µs)	Force (Kg f)
1100	-4.59	1236	-2.42	1368	-0.81	1504	0	1636	1.02	1772	3.15
1104	-4.54	1240	-2.36	1372	-0.77	1508	0	1640	1.08	1776	3.21
1108	-4.53	1244	-2.32	1376	-0.74	1512	0	1644	1.13	1780	3.27
1112	-4.48	1248	-2.25	1380	-0.7	1516	0	1648	1.18	1784	3.39
1116	-4.42	1252	-2.21	1384	-0.67	1520	0	1652	1.22	1788	3.46
1120	-4.35	1256	-2.16	1388	-0.64	1524	0	1656	1.28	1792	3.54
1124	-4.35	1260	-2.11	1392	-0.6	1528	0	1660	1.32	1796	3.63
1128	-4.17	1264	-2.04	1396	-0.56	1532	0.05	1664	1.38	1800	3.72
1132	-4.18	1268	-2	1400	-0.53	1536	0.07	1668	1.42	1804	3.79
1136	-4.11	1272	-1.94	1404	-0.49	1540	0.09	1672	1.49	1808	3.92
1140	-4.04	1276	-1.88	1408	-0.46	1544	0.12	1676	1.54	1812	4.02
1144	-3.99	1280	-1.81	1412	-0.43	1548	0.15	1680	1.6	1816	4.14
1148	-3.91	1284	-1.76	1416	-0.4	1552	0.18	1684	1.68	1820	4.21
1152	-3.86	1288	-1.72	1420	-0.37	1556	0.21	1688	1.74	1824	4.32
1156	-3.81	1292	-1.66	1424	-0.34	1560	0.25	1692	1.79	1828	4.41
1160	-3.71	1296	-1.61	1428	-0.3	1564	0.29	1696	1.84	1832	4.54
1164	-3.68	1300	-1.56	1432	-0.27	1568	0.32	1700	1.92	1836	4.67
1168	-3.56	1304	-1.52	1436	-0.24	1572	0.36	1704	1.97	1840	4.69
1172	-3.54	1308	-1.47	1440	-0.21	1576	0.39	1708	2.04	1844	4.86
1176	-3.42	1312	-1.42	1444	-0.18	1580	0.43	1712	2.08	1848	4.92
1180	-3.36	1316	-1.38	1448	-0.15	1584	0.47	1716	2.17	1852	5.02
1184	-3.27	1320	-1.33	1452	-0.13	1588	0.51	1720	2.22	1856	5.12
1188	-3.19	1324	-1.27	1456	-0.1	1592	0.55	1724	2.28	1860	5.21
1192	-3.09	1328	-1.22	1460	-0.08	1596	0.59	1728	2.36	1864	5.29
1196	-3.03	1332	-1.17	1464	-0.06	1600	0.63	1732	2.43	1868	5.38
1200	-2.96	1336	-1.13	1468	-0.05	1604	0.67	1736	2.5	1872	5.5
1204	-2.88	1340	-1.08	1472	0	1608	0.7	1740	2.59	1876	5.6
1208	-2.84	1344	-1.05	1476	0	1612	0.76	1744	2.64	1880	5.67
1212	-2.78	1348	-1.01	1480	0	1616	0.8	1748	2.74	1884	5.77
1216	-2.7	1352	-0.97	1484	0	1620	0.84	1752	2.79	1888	5.82
1220	-2.66	1356	-0.93	1488	0	1624	0.88	1756	2.87	1892	5.9
1224	-2.59	1360	-0.89	1492	0	1628	0.92	1760	2.93	1896	5.98
1228	-2.53	1364	-0.86	1496	0	1632	0.97	1764	3.01	1900	6.02
1232	-2.48			1500	0			1768	3.1		

Figure A.4: T200 Public Performance Data 10-20V, 18 V

Appendix B

Piranha EKF SymPy Script

In the application, the essential EKF functions such as the Kalman gain functions, are generated first using a computer algebra system (CAS) called `SymPy` and the results are saved. This `SymPy` script will take about 5 10 minutes on a modern PC to finish. EKF function blocks are saved into separated binary files using the Python `dill` tool.

```
## Piranha EKF Symbolic Computation Script
# Author: Xiahua Liu
# Last Modified Date: 10/13/2021
# Description: This script generates the EKF functions in numpy
# format and saves them in binary files with the dill library
# The EKF invented in this script has 13 states.
# This script takes around 5~10 minutes on a modern computer.

from sympy import *
from sympy.algebras.quaternion import Quaternion

# Dill the result functions to disks
import dill
dill.settings['recurse'] = True

## Define state variables
# IMU delta angle measurements in body axes - rad
dax, day, daz = symbols("da_x, da_y, da_z", real=True)
# IMU delta velocity measurements in body axes - m/sec
dvx, dvy, dvz = symbols("dv_x, dv_y, dv_z", real=True)
# Quaternions
q0, q1, q2, q3 = symbols("q_0, q_1, q_2, q_3", real=True)
```

```
# NED velocity - m/sec
vn, ve, vd = symbols("v_n, v_e, v_d", real=True)
# NED position - m
pn, pe, pd = symbols("p_n, p_e, p_d", real=True)
# IMU time step
dt = symbols("dt", real=True)
# NED gravity vector - m/sec^2
g = symbols("g", real=True)
# NED earth fixed magnetic field components - milligauss
magN, magE, magD = symbols("mag_N, mag_E, mag_D", real=True)

P = MatrixSymbol('P', 13, 13) # P matrix
Q = MatrixSymbol('Q', 13, 13) # Q matrix
R_mag = MatrixSymbol('R_mag', 1,1) # R mag
R_gps = MatrixSymbol('R_gps', 1,1) # R gps postion

# Define the state vector
stateVector = Matrix([q0, q1, q2, q3, vn, ve, vd, pn, pe, pd, magN, magE, magD])

# Meaured angle speed and acceleration
da = Matrix([dax, day, daz])
dv = Matrix([dvx, dvy, dvz])

# Quaternion vector & rotation matrix
quat = Quaternion(q0, q1, q2, q3)
Tbn = quat.to_rotation_matrix()

# NED Gravity calculation
gvec=Matrix([0,0,g], real=True)


## Predict Step
# Quaternion update equation
delQuat = Quaternion(1, 0.5*dax*dt, 0.5*day*dt, 0.5*daz*dt)
quatNew = quat.mul(delQuat)

# Velocity update equation
vNew = Matrix([vn, ve, vd])+gvec*dt+Tbn*dv*dt

# Position update equation
pNew = Matrix([pn, pe, pd])+Matrix([vn, ve, vd])*dt

# Earth magnetic field update equation
magNew = Matrix([magN, magE, magD])

# Process equation
processEqn = Matrix([quatNew.a, quatNew.b, quatNew.c, quatNew.d, vNew, pNew,
    magNew])
```

```
## Innovation Step
# Linearized state transition matrix
F=simplify(processEqn.jacobian(stateVector))

# Pack into lambda objects
F_lambda=lambdify((q0,q1,q2,q3,dvx,dvy,dvz,dax,day,daz,dt), F, 'numpy')
# Dill the lambda functions
dill.dump(F_lambda, open("F_lambda.npf", "wb"))

# New covariance matrix
PP=simplify(F*P*F.T+Q)

# Pack into lambda objects
PP_lambda=lambdify((q0,q1,q2,q3,dvx,dvy,dvz,dax,day,daz,dt, P, Q, g), PP, 'numpy')
# Dill the lambda functions
dill.dump(PP_lambda, open("PP_lambda.npf", "wb"))


## Mag measurement fusion
magMeas=Tbn.T*Matrix([magN, magE, magD])

# Mag measurement jacobian
H_mag=simplify(magMeas.jacobian(stateVector))

# Mag observation funciton
H_magx=H_mag[0,:]
H_magy=H_mag[1,:]
H_magz=H_mag[2,:]

# Kalman gains for mag fustion
K_mx=simplify((P*H_magx.T)/(H_magx*P*H_magx.T+R_mag))
K_my=simplify((P*H_magy.T)/(H_magy*P*H_magy.T+R_mag))
K_mz=simplify((P*H_magz.T)/(H_magz*P*H_magz.T+R_mag))

# GPS measurement jacobian
H_gps=simplify(Matrix([pn, pe, pd]).jacobian(stateVector))

# GPS observation function
H_gpsn=H_gps[0,:]
H_gpse=H_gps[1,:]
H_gpsd=H_gps[2,:]

# Kalman gains for GPS fustion
K_pn=simplify((P*H_gpsn.T)/(H_gpsn*P*H_gpsn.T+R_gps))
K_pe=simplify((P*H_gpse.T)/(H_gpse*P*H_gpse.T+R_gps))
K_pd=simplify((P*H_gpsd.T)/(H_gpsd*P*H_gpsd.T+R_gps))
```

```
## Pack result function into lambda objects
# Pack into lambda objects
H_magx_lambda=lambdify((q0,q1,q2,q3, magN, magE, magD), H_magx, 'numpy')
H_magy_lambda=lambdify((q0,q1,q2,q3, magN, magE, magD), H_magy, 'numpy')
H_magz_lambda=lambdify((q0,q1,q2,q3, magN, magE, magD), H_magz, 'numpy')

# Dill the lambda functions
dill.dump(H_magx_lambda, open("H_magx_lambda.npf", "wb"))
dill.dump(H_magy_lambda, open("H_magy_lambda.npf", "wb"))
dill.dump(H_magz_lambda, open("H_magz_lambda.npf", "wb"))

# Pack into lambda objects
H_gpsn_lambda=lambdify((), H_gpsn, 'numpy')
H_gpse_lambda=lambdify((), H_gpse, 'numpy')
H_gpsd_lambda=lambdify((), H_gpsd, 'numpy')

# Dill the lambda functions
dill.dump(H_gpsn_lambda, open("H_gpsn_lambda.npf", "wb"))
dill.dump(H_gpse_lambda, open("H_gpse_lambda.npf", "wb"))
dill.dump(H_gpsd_lambda, open("H_gpsd_lambda.npf", "wb"))

# Pack into lambda objects
K_mx_lambda=lambdify((q0,q1,q2,q3,magN,magE,magD, P, R_mag), K_mx, 'numpy')
K_my_lambda=lambdify((q0,q1,q2,q3,magN,magE,magD, P, R_mag), K_my, 'numpy')
K_mz_lambda=lambdify((q0,q1,q2,q3,magN,magE,magD, P, R_mag), K_mz, 'numpy')

# Dill the lambda functions
dill.dump(K_mx_lambda, open("K_mx_lambda.npf", "wb"))
dill.dump(K_my_lambda, open("K_my_lambda.npf", "wb"))
dill.dump(K_mz_lambda, open("K_mz_lambda.npf", "wb"))

# Pack into lambda objects
K_pn_lambda=lambdify((P,R_gps), K_pn, 'numpy')
K_pe_lambda=lambdify((P,R_gps), K_pe, 'numpy')
K_pd_lambda=lambdify((P,R_gps), K_pd, 'numpy')

# Dill the lambda functions
dill.dump(K_pn_lambda, open("K_pn_lambda.npf", "wb"))
dill.dump(K_pe_lambda, open("K_pe_lambda.npf", "wb"))
dill.dump(K_pd_lambda, open("K_pd_lambda.npf", "wb"))
```

Bibliography

[1] Industry Agenda. The new plastics economy rethinking the future of plastics, 2016.

[2] John E Bortz. A new mathematical formulation for strapdown inertial navigation. *IEEE transactions on aerospace and electronic systems*, AES-7(1):61–66, 1971.

[3] Ali Chamas, Hyunjin Moon, Jiajia Zheng, Yang Qiu, Tarnuma Tabassum, Jun Hee Jang, Mahdi Abu-Omar, Susannah L. Scott, and Sangwon Suh. Degradation rates of plastics in the environment. *ACS Sustainable Chemistry & Engineering*, 8(9):3494–3511, 2020.

[4] Andres Cozar, Fidel Echevarria, J. Ignacio Gonzalez-Gordillo, Xabier Irigoien, Barbara Ubeda, Santiago Hernandez-Leon, Alvaro T. Palma, Sandra Navarro, Juan Garcia-de Lomas, Andrea Ruiz, Maria L. Fernandez-de Puelles, and Carlos M. Duarte. Plastic debris in the open ocean. *Proceedings of the National Academy of Sciences*, 111(28):10239–10244, 2014.

[5] Yunduan Cui, Shigeki Osaki, and Takamitsu Matsubara. Autonomous boat driving system using sample-efficient model predictive control-based reinforcement learning approach. *Journal of Field Robotics*, pages 1–24, 09 2020.

[6] Peter Davison and Rebecca Asch. Plastic ingestion by mesopelagic fishes in the north pacific subtropical gyre. *Marine Ecology Progress Series*, 432:173–180, 01 2013.

[7] C.G. Källström K.J. Åström. Identification of ship steering dynamics. *Automatica*, 12(1):9–22, 1976.

[8] L. Lebreton, B. Slat, F. Ferrari, B. Sainte-Rose, J. Aitken, R. Marthouse, S. Hajbane, S. Cunsolo, A. Schwarz, A. Levivier, K. Noble, P. Debeljak, H. Maral, R. Schoeneich-Argent, R. Brambini, and J. Reisser. Evidence that the great pacific garbage patch is rapidly accumulating plastic. *Scientific Reports*, 8(1):4666, 3 2018.

[9] Wasif Naeem, Rhondasutton Sutton, and John Chudley. Modelling and control of an unmanned surface vehicle for environmental monitoring. *United Kingdom Automatic Control Council*, 10 2021.

[10] Meyer Nahon. A simplified dynamics model for autonomous underwater vehicles. In *Proceedings of Symposium on Autonomous Underwater Vehicle Technology*, pages 373–379. IEEE, 1996.

[11] Umber Noreen, Ahcène Bounceur, and Laurent Clavier. A study of lora low power and wide area network technology. In *2017 International Conference on Advanced Technologies for Signal and Image Processing (ATSIP)*, pages 1–6, 2017.

[12] Hannah Ritchie and Max Roser. Plastic pollution. *Our World in Data*, 2018. https://ourworldindata.org/plastic-pollution.

[13] Paul G Savage. Strapdown inertial navigation integration algorithm design part 1: Attitude algorithms. *Journal of guidance, control, and dynamics*, 21(1):19–28, 1998.

[14] Paul G Savage. Strapdown inertial navigation integration algorithm design part 2: Velocity and position algorithms. *Journal of Guidance, Control, and dynamics*, 21(2):208–221, 1998.

[15] Emma Schmaltz, Emily C. Melvin, Zoie Diana, Ella F. Gunady, Daniel Rittschof, Jason A. Somarelli, John Virdin, and Meagan M. Dunphy-Daly. Plastic pollution solutions: emerging technologies to prevent and collectmarineplastic pollution. *Environment International*, 144:106067, 2020.

[16] Christian R Sonnenburg and Craig A Woolsey. Modeling, identification, and control of an unmanned surface vehicle. *Journal of Field Robotics*, 30(3):371–398, 2013.

[17] Bao-qiang Tian, Jian-cheng Yu, and Ai-qun Zhang. Dynamic modeling of wave driven unmanned surface vehicle in longitudinal profile based on dh approach. *Journal of Central South University*, 22(12):4578–4584, 2015.

[18] TOSHIBA. *MOSFETs Silicon N-channel MOS (U-MOSIX-H) TK4R3E06PL*, 2021. Rev.3.0.

[19] Joohyun Woo, Jongyoung Park, Chanwoo Yu, and Nakwan Kim. Dynamic model identification of unmanned surface vehicles using deep learning network. *Applied Ocean Research*, 78:123–133, 2018.